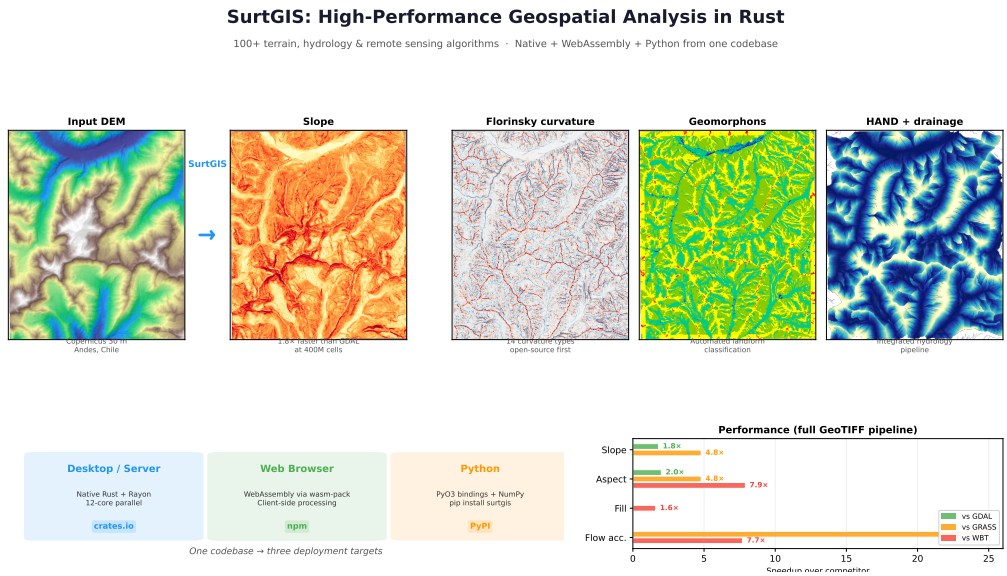
Graphical Abstract

SurtGIS: A High-Performance Raster Geospatial Analysis Library in Rust with WebAssembly and Python Support

Francisco Parra



Highlights

SurtGIS: A High-Performance Raster Geospatial Analysis Library in Rust with WebAssembly and Python Support

Francisco Parra

- Rust library with 127 terrain, hydrology, and imagery algorithms
- Compiles to native, WebAssembly, and Python from one codebase
- Open-source implementation of Florinsky's 14-curvature system
- 1.8× faster than GDAL for slope; 23× for flow accumulation vs GRASS
- Numerical accuracy within 10^{-5} degrees of analytical solutions

SurtGIS: A High-Performance Raster Geospatial Analysis Library in Rust with WebAssembly and Python Support

Francisco Parra[a,*]

[a]*Departamento de Ingeniería Informática, Universidad de Santiago de Chile, Santiago, Chile*

Abstract

We present SurtGIS, an open-source raster geospatial analysis library in Rust providing 127 terrain, hydrological, and remote sensing algorithms. The library compiles to native code with automatic parallelization via Rayon, to WebAssembly for browser-based applications, and exposes Python bindings through PyO3. Key contributions include Florinsky's complete 14-curvature morphometric system, three depression handling algorithms, and four flow direction methods. Benchmarks on synthetic DEMs up to $20{,}000^2$ cells demonstrate speedups on full GeoTIFF pipelines: 1.7–1.8× over GDAL and 4.5–4.9× over GRASS GIS for slope; 2.0× over GDAL for aspect; and 7.5–23.1× for D8 flow accumulation. WebAssembly adds 10–32× overhead versus native Rust but enables browser-based terrain analysis unavailable in existing tools. Numerical accuracy is within 5.7×10^{-5} degrees RMSE against analytical solutions.

Keywords: terrain analysis, digital elevation model, hydrology,

*Corresponding author

Email address: francisco.parra.o@usach.cl (Francisco Parra)

WebAssembly, Rust, geomorphometry

1. Introduction

Digital terrain analysis is fundamental to environmental modeling, encompassing slope stability assessment, hydrological network extraction, landform classification, and habitat suitability mapping (Florinsky, 2025). The increasing availability of high-resolution elevation data—from lidar surveys at sub-meter resolution to global datasets such as SRTM (Farr et al., 2007) and Copernicus DEM at 30 m—has created demand for efficient, accessible, and reproducible geospatial processing tools.

Current geospatial software presents researchers with a trade-off between performance and accessibility. Python-based tools such as rasterio and richdem offer ease of use but are constrained by interpreter overhead for pixel-level computation. Compiled libraries such as GDAL (GDAL/OGR contributors, 2024) provide performance but lack comprehensive terrain analysis algorithms and cannot execute in browser environments. Specialized tools such as WhiteboxTools (Lindsay, 2016b) and GRASS GIS (Neteler et al., 2012) offer extensive algorithm collections but are either partially proprietary or require heavyweight installations unsuitable for web deployment. RichDEM (Barnes, 2016) provides efficient C++ implementations of hydrological algorithms with Python bindings, SAGA GIS (Conrad et al., 2015) offers a comprehensive suite of terrain analysis modules, and TauDEM (Tarboton, 2015) provides the reference D-infinity flow implementation (Tarboton, 1997) via MPI-based parallelism. In the Python ecosystem, pysheds (Bartos, 2020) and pyflwdir (Eilander et al., 2021) provide lightweight hydrological routing but

lack terrain morphometry and browser deployment. TauDEM is included in the hydrological benchmarks as the representative MPI-based parallel geocomputation framework, executed on the same machine across 1–16 MPI processes (Section 5.5).

Cloud-native geospatial formats—Cloud-Optimized GeoTIFF (COG), SpatioTemporal Asset Catalogs (STAC; Radiant Earth Foundation, 2021), and Zarr—and the platforms that publish them (Gorelick et al., 2017) have shifted the dominant access pattern from full-file download to on-demand spatial-range reads over HTTP. This benefits any client capable of issuing HTTP byte-range requests: batch processing services, scientific workflows, and increasingly client-side applications running in browsers. The browser case in particular requires libraries that can execute computations on the client without round-tripping data to a server, which motivates WebAssembly (Wasm) as a compilation target (Haas et al., 2017); existing comprehensive raster analysis libraries do not provide a Wasm build.

We present SurtGIS, an open-source raster geospatial analysis library implemented in Rust that addresses these limitations. Rust is the implementation language because it satisfies three constraints simultaneously: (i) memory safety without garbage collection (Perkel, 2020), allowing predictable latency and cache-friendly memory layout that interpreted languages cannot guarantee; (ii) zero-cost abstractions yielding C-level performance for tight numerical loops; and (iii) first-class WebAssembly compilation, which enables identical algorithms to run client-side without the runtime overhead and fork-based parallelism that CPython requires for shared-memory work. Python users access the same engine without rewriting workflows:

the `surtgis-python` package is a PyO3 binding over the Rust core, installable via `pip install surtgis` so that the speedups reported here apply to Python notebook usage at the cost of a single zero-copy buffer view at the language boundary. The library provides:

1. 127 distinct computational operations for terrain analysis, hydrology, remote sensing, landscape ecology, and texture analysis (complete inventory and counting methodology in Supplementary Table S1);
2. An open-source implementation of Florinsky's complete 14-curvature morphometric system (Florinsky, 2025);
3. A unified codebase that compiles to native (with Rayon parallelization), WebAssembly (sequential fallback), and Python (via PyO3);
4. Cloud-native data access through integrated STAC client and COG reader.

This paper is structured as follows: Section 2 describes the software architecture and algorithms; Section 3 provides an illustrative example; Section 4 presents an application example; Section 5 presents the performance evaluation; Section 6 compares with existing tools; and Section 7 discusses impact and conclusions.

2. Software description

2.1. Architecture

SurtGIS is organized as a Rust workspace of nine crates organised in three layers (Figure 1): an I/O layer (`surtgis-core`: raster data structures, native GeoTIFF I/O, NoData handling; `surtgis-cloud`: STAC client,

COG reader, Zarr/NetCDF/GRIB readers behind feature flags), a compute layer (`surtgis-algorithms`: all geospatial algorithms; `surtgis-parallel`: Rayon-based parallel adapters; `surtgis-colormap`: color schemes and raster rendering), and a bindings layer (`surtgis-cli`: command-line interface; `surtgis-wasm`: WebAssembly bindings; `surtgis-python`: PyO3 bindings; `surtgis-gui`: optional egui desktop application, not benchmarked here). This modular structure allows users—and downstream binaries—to depend only on the layers they need: a Python notebook user pulls in `surtgis-python`, which in turn pulls `surtgis-algorithms` and `surtgis-core`, with no run-time dependency on the cloud or GUI crates.

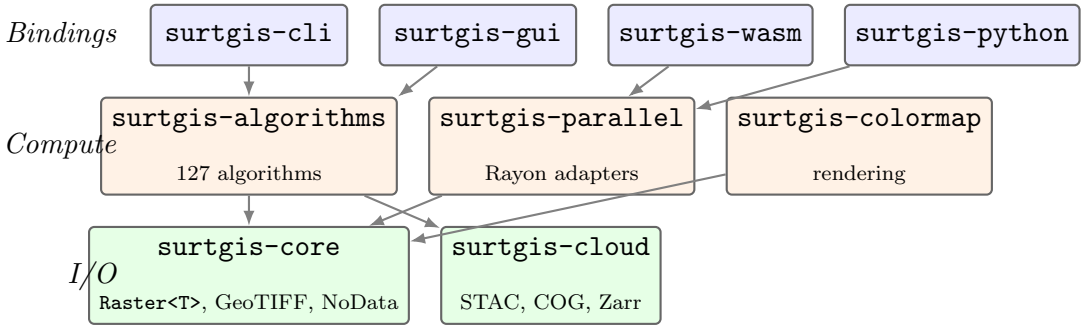


Figure 1: Layered architecture of the SurtGIS workspace. Arrows indicate compile-time crate dependencies. The I/O layer provides raster data structures and GeoTIFF / cloud-format readers; the compute layer hosts all algorithms and the parallel-execution adapters; the bindings layer publishes the same algorithm surface to four execution targets (CLI, desktop GUI, browser via WebAssembly, and Python via PyO3). Every binding consumes the same compute layer, so a slope computation in a Python notebook executes the same Rust function as the CLI.

`surtgis-algorithms` is itself partitioned into nine algorithm submodules organised by domain (Table 1), plus utility submodules for vector I/O and

temporal-stack helpers that are not enumerated as algorithms. The submodules share a common `Raster<T>` datum, the parallel-execution adapter exposed by `surtgis-parallel`, and a `maybe_rayon` compile-time pattern that selects between Rayon's parallel iterators—based on work-stealing scheduling (Blumofe and Leiserson, 1999)—on native targets and sequential fallback iterators on WebAssembly. Each algorithm is therefore written once and benefits from multi-core parallelism wherever available, without code duplication across deployment targets.

The raster data model stores elevation and derived grids as contiguous `ndarray::Array2<f64>` arrays with associated metadata (cell size, NoData value, spatial reference, affine transform). This in-memory representation enables efficient cache-friendly access patterns for moving-window operations that dominate terrain analysis workloads, though it requires holding the full dataset in RAM (see Section 5.1 for memory implications).

Data robustness. All algorithms propagate NoData values: boundary cells and neighbors of NoData pixels produce NoData in the output rather than artifacts. Non-square pixels (different Δx and Δy) are supported via the affine transform metadata; slope and curvature computations use per-axis cell sizes. Input validation uses Rust's `Result` type throughout the API, with explicit error types for invalid rasters (empty, non-finite values), unsupported CRS, and I/O failures. **Important:** geographic CRS inputs (e.g., WGS84 in degrees) are currently processed in degree units, which produces physically meaningless slope and curvature values; users must reproject to a metric CRS (e.g., UTM) before analysis. NaN and infinity values in input rasters are treated as NoData.

Table 1: Algorithm submodules of `surtgis-algorithms`, with the parallel decomposition pattern (Section 2.2) used by the majority of the submodule's algorithms. "Streaming" indicates whether at least one algorithm in the submodule supports out-of-core execution.

Submodule	Count	Representative algorithms	Predominant pattern	Streaming
`terrain`	49	slope, aspect, hillshade, Florinsky 14 curvatures, TPI, TRI, geomorphons, MRVBF, openness, viewshed, SVF, solar radiation	focal (halo)	yes
`hydrology`	19	Planchon–Darboux / Priority-Flood / breach depression handling, D8/MFD/D-infinity/TFGA flow direction and accumulation, TWI, HAND	dependency-aware	no
`imagery`	22	NDVI, NDWI, EVI, BSI, NBR, band-math, reclassify, threshold, change-vector analysis	cell-wise	yes
`statistics`	11	focal stats (mean / std / range / median / min / max / sum / majority), zonal stats, Moran's I, Getis–Ord G_i^*	global reduction	yes
`interpolation`	9	IDW, nearest neighbour, TIN, natural neighbour, thin-plate spline, ordinary / universal / regres-	cell-wise / reduction	no

2.2. Parallel strategies

SurtGIS targets single-node throughput with cloud-native streaming I/O, a complementary point in the design space to distributed-memory raster libraries such as pRPL (Guan and Clarke, 2010), pRPL+GTIOL (Miao et al., 2017), and the parallel raster-based geocomputation operator frameworks of Qin et al. (2014), which target cluster execution under the Message Passing Interface (MPI). For precision of terminology: MPI is a message-passing standard and library ecosystem for distributed-memory programming, whereas massively parallel processing (MPP) denotes a class of system architectures on which such libraries commonly run; the two belong to different technical tiers and are not used interchangeably here. Single-node decomposition is appropriate for the deployment surfaces SurtGIS targets—desktop and laptop machines, single virtual machines, web browsers via WebAssembly, and Python notebooks—because the data volumes those surfaces can accommodate (DEMs up to $20K^2$ in our benchmarks, individual Sentinel-2 scenes, single watersheds) are tractable on one machine when paired with streaming GeoTIFF I/O and HTTP range reads against cloud object storage. Distributed-memory parallelism via MPI is not displaced by SurtGIS but is a distinct deployment regime; the strategies described below characterise within-node decomposition, scheduling, and aggregation, and Section 5.5 provides a direct single-node benchmark against TauDEM as the MPI representative.

2.2.1. Decomposition patterns

SurtGIS algorithms fall into five within-node decomposition patterns (Table 2). The pattern dictates how the input domain is partitioned across

worker threads, how partial results are combined, and how (if at all) workers communicate.

Domain decomposition. The default partition is by row strip: a Rayon `par_iter` over chunks of the row index splits an $H \times W$ grid into roughly $\lceil H/p \rceil$-row strips per worker, where p is the number of threads. Row strips are preferred over square tiles for three reasons: (i) GeoTIFF files store data row-wise on disk, so row-aligned access patterns minimise seeks and maximise sequential I/O bandwidth, mirroring the design rationale of pRPL+GTIOL (Miao et al., 2017); (ii) for focal kernels with halo k (typically $k = 1$ for Horn-method slope and aspect, or $k = 10$ for radius-10 TPI), a row strip requires only $2k$ extra rows per worker for boundary reads, whereas square tiles require $O(k\sqrt{N/p})$ halo cells; (iii) row strips reuse memory in contiguous f64 array slices, which the Rust compiler and the CPU prefetcher can vectorise without explicit SIMD intrinsics.

Result aggregation. For cell-wise and focal patterns, each worker writes into a disjoint slice of the output buffer; no aggregation is needed and no synchronisation primitive is acquired during the parallel section. For global-reduction algorithms, Rayon's `reduce` combinator merges per-strip partials in $O(\log p)$ tree time after the parallel map phase. Dependency-aware algorithms (Priority-Flood, D8 flow accumulation) cannot be fully domain-decomposed without changing the per-cell algorithm; the v0.7 release uses serial or partially-parallel implementations whose Amdahl-bound behaviour is empirically characterised in Section 5.4. Tile-and-reconcile parallel variants of Barnes (2017) are on the SurtGIS roadmap but are not part of this evaluation.

Table 2: Parallel decomposition patterns used in SurtGIS, organised by algorithm class. "Halo" = number of input rows shared between adjacent worker strips for boundary access; "Reduction" = whether worker outputs require global combination. Empirical scaling results for the focal, dependency-aware, and global-reduction classes are reported in Section 5.4.

Pattern	Representative algorithms	Decomposition	Halo	Reduction
Cell-wise	NDVI, band-math, reclassify, threshold	Row strip	0	no
Focal (halo)	Slope, aspect, hillshade, Florinsky curvatures, TPI, TRI	Row strip	k rows	no
Streaming	Twelve terrain algorithms (out-of-core variants)	Disk strip	k rows	no
Dependency-aware	Priority-Flood fill, D8 flow accumulation	Whole-domain	—	no
Global reduction	Focal/zonal statistics, Moran's I, Getis–Ord G_i^*	Per-strip partials	0	yes

Inter-thread communication. The decomposition is structured so that workers do not communicate during the parallel section. Halo rows are read-only: the input array is shared via Rust's borrow checker as an immutable reference, and each worker reads from the input strip plus its halo without ever writing to a region another worker reads. The output buffer is accessed via mutually disjoint mutable slices (`split_at_mut`), an aliasing-free pattern verified at compile time. There are no atomic operations, mutexes, or message passes inside the inner loops of any cell-wise or focal kernel. This contrasts with shared-mutable patterns common in OpenMP raster code, where atomic increments on a shared accumulator can dominate at high p.

Task scheduling and load balancing. Rayon scheduling uses work-stealing (Blumofe and Leiserson, 1999): each worker has a deque of tasks and idle workers steal from busy peers. SurtGIS does not pin threads to cores, leaving placement to the OS scheduler; on the hybrid Intel i7-1270P used in our benchmarks (4 P-cores + 8 E-cores), this means performance and efficiency cores receive work according to availability rather than a fixed split. For uniform-work patterns (cell-wise and focal), the row-strip decomposition assigns each worker an equal cell count, so static load balance is intrinsic to the partition and the work-stealing layer absorbs only the residual variance from OS interrupts and cache effects. Section 5.4 confirms this empirically: focal-kernel efficiency curves do not exhibit imbalance signatures (sharp drops at specific p), only the smooth bandwidth-bound saturation expected from a well-balanced memory-bound workload.

Geocomputational heterogeneity. Non-uniform per-cell work can arise when (a) cells contain NoData and short-circuit the inner kernel, (b) flow-routing

algorithms have data-dependent recursion depth, or (c) viewshed and visibility computations have variable ray cardinalities depending on local relief. For (a), the kernel cost of skipping a NoData cell is approximately 10% of a full kernel, so the worst-case imbalance between a strip with 100% NoData and a strip with 100% data is $\sim 10\times$; Rayon's dynamic chunk sizing absorbs this by re-distributing fragments of strips to idle workers. For (b) and (c), SurtGIS uses task-level rather than data-level parallelism (e.g., viewshed-from-many-points dispatches one Rayon task per observer; multi-scale curvature dispatches one task per scale), recovering near-linear scaling whenever the algorithm admits multiple independent task instances. The cell-level partition is only chosen when the per-cell work is approximately uniform.

Static decomposition versus dynamic work-stealing. The parallel geocomputation literature has developed sophisticated *static* domain decomposition strategies—spatially coherent aggregation of adjacent subtasks and complexity-driven partition sizing informed by per-region workload estimates (Qin et al., 2014; Wang et al., 2021). These strategies are essential in the distributed-memory (MPI) setting for which they were designed: the initial decomposition there is fixed for the lifetime of the computation, migrating work between processes is expensive, and a poor initial partition therefore determines the achievable load balance. A shared-memory work-stealing runtime occupies a different point in this design space. Rayon divides the domain into sub-ranges substantially finer than the worker count, and idle threads steal queued sub-ranges from busy peers at a cost of nanoseconds rather than message-passing latencies; the load-balancing problem that complexity-driven static partitioning solves *a priori* from properties of the input raster is thus solved *at runtime*

by the scheduler—including for imbalance sources that no static analysis of the input can predict, such as data-dependent branch costs and interference from other processes on the machine. The two approaches are therefore complements attached to different memory models rather than competing alternatives at the same tier: for the reprojection-corner NoData pattern—the worst case for a static row-strip partition, in which entire strips near a rotated grid's edges are dominated by short-circuiting NoData cells—the work-stealing scheduler redistributes the surviving sub-ranges to idle threads at runtime, so no static, complexity-aware partition of the input is required to keep cores busy. For globally coupled algorithms (Priority-Flood, watershed labelling, flow accumulation), where neither fine-grained stealing nor static partitioning applies directly, decomposition is genuinely harder; the static tile-and-reconcile strategies of the cited literature remain the appropriate foundation for a future distributed-memory tier of SurtGIS (Section 7).

Streaming. For DEMs that exceed available RAM, twelve terrain algorithms implement an out-of-core streaming variant that reads, computes, and writes a single strip at a time, retaining only $2k$ halo rows between iterations (where k is the kernel radius). The streaming variant uses the same row-strip parallel decomposition within each loaded strip, so the speedup characterisation in Section 5.4 carries through. The disk I/O itself is serial in v0.7 (read-then-compute-then-write per strip); concurrent read-while-compute pipelining is identified as future work.

2.3. Terrain analysis algorithms

SurtGIS implements a comprehensive set of terrain analysis algorithms organized into several categories (Table 3).

Table 3: Terrain analysis algorithms implemented in SurtGIS.

Category	Algorithms
Basic derivatives	Slope, aspect, northness, eastness
Hillshading	Analytical hillshade, multidirectional hillshade
Curvature (classical)	Profile, plan, general, tangential curvature
Curvature (Florinsky)	14 types: H, K, M, E, $k_{\min}$, $k_{\max}$, k_h, k_v, k_{he}, k_{ve}, K_a, K_r, rotor, Laplacian
Landform indices	TPI, TRI, DEV, VRM, geomorphons, MRVBF/MRRTF
Visibility	Viewshed (R2, XDraw, PDERL), SVF, positive/negative openness
Multiresolution	Gaussian scale-space, Chebyshev spectral derivatives, 2D-SSA
Other	Convergence index, wind exposure, solar radiation, contour lines, cost distance, uncertainty propagation

2.3.1. Slope and aspect

SurtGIS computes slope and aspect using Horn's method (Horn, 1981), which estimates first-order partial derivatives from a 3×3 moving window using a weighted finite-difference scheme. The slope gradient is:

$$\text{slope} = \arctan\sqrt{\left(\frac{\partial z}{\partial x}\right)^2 + \left(\frac{\partial z}{\partial y}\right)^2} \tag{1}$$

where the partial derivatives are estimated by the Horn (1981) 3×3 kernel:

$$\frac{\partial z}{\partial x} = \frac{(z_{+1,-1} + 2z_{+1,0} + z_{+1,+1}) - (z_{-1,-1} + 2z_{-1,0} + z_{-1,+1})}{8\Delta x} \tag{2}$$

(and analogously for $\partial z/\partial y$), using weights $[1, 2, 1]$ for the central row/column. This is the same method used by GDAL and GRASS GIS. WhiteboxTools uses a 3rd-order bivariate Taylor polynomial on a 5×5 neighborhood (Florinsky, 2016), which is more robust against noise but produces systematically different slope estimates than Horn's 3×3 kernel; this methodological difference accounts for the cross-tool deviations reported in Section 5.2.

2.3.2. Florinsky's 14-curvature system

A key contribution of SurtGIS is the open-source implementation of the complete morphometric variable system described by Florinsky (2025). Surface curvatures are computed from partial derivatives estimated via the Evans–Young method (Evans, 1972) on a 3×3 moving window. A local bivariate polynomial $z = rx^2/2 + sxy + ty^2/2 + px + qy + z_0$ is fitted by least squares to the nine elevation values in the window, yielding the first-order ($p = \partial z/\partial x$, $q = \partial z/\partial y$) and second-order ($r = \partial^2 z/\partial x^2$, $s = \partial^2 z/\partial x \partial y$, $t = \partial^2 z/\partial y^2$) partial derivatives. From these, the 14 curvature types are derived:

$$H = -\frac{(1+q^2)r - 2pqs + (1+p^2)t}{2(1+p^2+q^2)^{3/2}} \tag{3}$$

$$K = \frac{rt - s^2}{(1+p^2+q^2)^2} \tag{4}$$

where H is mean curvature and K is Gaussian curvature. Note that the sign convention for H (Eq. 3) follows Florinsky (2025): the negative sign in the numerator yields $H > 0$ for convex surfaces (ridges) and $H < 0$ for concave surfaces (valleys). Some references omit this negation, producing the opposite convention; users should verify sign semantics when comparing across tools. The remaining 12 curvatures (unsphericity M, difference E, principal curvatures $k_{\min}$ and $k_{\max}$, horizontal k_h, vertical k_v, excess curvatures k_{he} and k_{ve}, accumulation K_a, ring K_r, rotor, and Laplacian) are derived from combinations of H, K, and the partial derivatives. SAGA GIS implements a subset (profile, plan, tangential, longitudinal, minimal, maximal curvature) based on earlier Florinsky work, but does not include the excess curvatures (k_{he}, k_{ve}), accumulation (K_a), ring (K_r), unsphericity (M), or difference (E) curvatures from the 2025 formulation. A comprehensive taxonomy of surface curvatures and their physical interpretation is provided by Minár et al. (2020). To our knowledge—based on a survey of GDAL, GRASS, SAGA, WhiteboxTools, RichDEM, and TauDEM documentation as of February 2026—SurtGIS is the first open-source library to implement the complete 14-type system as defined by Florinsky (2025). We acknowledge that the book's recent publication date (2025) means other implementations may emerge.

2.3.3. Geomorphons

The geomorphon classification algorithm (Jasiewicz and Stepinski, 2013) identifies landform elements by analyzing the ternary pattern of elevation differences along eight cardinal and diagonal directions within a search radius. SurtGIS implements this with configurable flatness threshold and lookup

radius, producing a 10-class landform map (flat, peak, ridge, shoulder, spur, slope, hollow, footslope, valley, pit).

2.4. Hydrological algorithms

The hydrology module provides a complete pipeline from raw DEM to derived hydrological indices (Table 4).

Table 4: Hydrological algorithms implemented in SurtGIS.

Stage	Algorithms
Depression handling	Planchon–Darboux filling (Planchon and Darboux, 2001), Priority-Flood (Barnes et al., 2014), breach depressions (Lindsay, 2016a)
Flow direction	D8 (O'Callaghan and Mark, 1984), MFD (Quinn et al., 1991), D-infinity (Tarboton, 1997), TFGA (Li et al., 2024)
Flow accumulation	D8, MFD, D-infinity accumulation
Derived indices	TWI (Kopecký et al., 2021), SPI ($= A_s \tan\beta$), STI ($= (A_s/22.13)^{0.6}(\sin\beta/0.0896)^{1.3}$) (Wilson and Gallant, 2000), HAND (Nobre et al., 2011)[b]
Network extraction	Stream network, watershed delineation[a]

[a]Watershed delineation and stream network extraction are implemented but not benchmarked; these are downstream operations that depend on the flow direction grid and are dominated by I/O rather than algorithmic computation. [b]A_s denotes specific catchment area (contributing area per unit contour width, m^2/m); β is the local slope angle.

Depression handling is a critical preprocessing step. SurtGIS implements

three approaches: (1) the Planchon–Darboux filling algorithm (Planchon and Darboux, 2001), which produces flat areas that can affect flow routing; (2) the Priority-Flood algorithm (Barnes et al., 2014), an optimal $O(N \log N)$ method using a priority queue; and (3) the breach-based approach of Lindsay (2016a), which carves channels through barriers to maintain more natural flow paths.

For flow direction, the D8 algorithm (O'Callaghan and Mark, 1984) assigns flow to the single steepest downslope neighbor. The Multiple Flow Direction (MFD) algorithm of Quinn et al. (1991) distributes flow proportionally to all downslope neighbors based on slope, better representing sheet flow on gentle terrain. The D-infinity algorithm of Tarboton (1997), implemented in TauDEM (Tarboton, 2015), uses triangular facets to determine a continuous flow angle, partitioning flow between the two cells that bracket this angle.

2.5. Additional modules

Beyond terrain and hydrology, SurtGIS includes modules for remote sensing (22 algorithms: 16 spectral indices including NDVI, NDWI, EVI, BSI, NBR, plus band-math, reclassify, raster difference, change-vector analysis), landscape ecology (Shannon and Simpson diversity, patch density), texture analysis (GLCM Haralick features and Sobel edge detection), classification (PCA, k-means, ISODATA, minimum distance, maximum likelihood), morphology (erosion, dilation, opening, closing, gradient, top-hat, black-hat), interpolation (IDW, nearest neighbour, TIN, natural neighbour, thin-plate spline, ordinary / universal / regression kriging), and statistics (focal stats, zonal stats, Moran's I, Getis–Ord G_i^*). These modules share the same cross-

platform compilation model but are not the focus of the benchmarks presented here; a complete inventory of all 127 algorithms is provided in Supplementary Table S1.

2.6. Cross-platform deployment

SurtGIS compiles to three deployment targets from the same codebase:

Native (desktop/server). Compiled with Rayon for automatic data parallelism. Published as Rust crates on `https://crates.io`.

Python. Bindings via PyO3 (PyO3 Project Contributors, 2024) expose 97 functions covering terrain, hydrology, imagery, morphology, statistics, interpolation, classification, landscape, and texture with NumPy array input/output. Functions are organized under a flat `surtgis` namespace (e.g., `surtgis.slope()`, `surtgis.advanced_curvature()`); a submodule structure (`surtgis.terrain`, `surtgis.hydrology`) is planned for a future release. Data transfer uses NumPy's buffer protocol for zero-copy access where possible; when the Rust function requires owned data, a single `memcpy` is performed. Published on PyPI as the `surtgis` package, installable via `pip install surtgis`. Prebuilt wheels are produced by the `python-release.yml` GitHub Actions workflow for Linux (x86_64, aarch64; manylinux and musllinux), macOS (x86_64, ARM64), and Windows (x86_64) across CPython 3.9–3.13. The remaining 30 algorithms are progressively exposed in each release, with full Python coverage targeted for version 1.0; all 127 are accessible via the native Rust API.

WebAssembly. Compiled via `wasm-pack` with JavaScript/TypeScript bindings. Published on npm as the `surtgis` package. Enables terrain analysis directly in web browsers without server-side processing.

3. Illustrative example

To demonstrate SurtGIS's capabilities, we present a terrain characterization workflow in Python. The example synthesizes a small DEM in NumPy, computes terrain derivatives, and extracts hydrological features. The synthetic DEM is fully reproducible (fixed random seed) so the listing runs end-to-end without external data dependencies:

```
import numpy as np
import surtgis

# Synthesize a 200x200 DEM at 30 m resolution: a Gaussian
# hill superimposed on multi-octave deterministic noise.
n, cell_size = 200, 30.0
y, x = np.mgrid[0:n, 0:n] * cell_size
hill = 1500 * np.exp(-((x - n*cell_size/2)**2 +
                       (y - n*cell_size/2)**2) / (1500**2))
rng = np.random.default_rng(seed=42)
noise = sum((1.0 / 2**k) * rng.standard_normal((n, n))
            for k in range(1, 5))
dem = (hill + 200 * noise).astype(np.float64)

# Terrain derivatives
```

```
slope = surtgis.slope(dem, cell_size)
aspect = surtgis.aspect_degrees(dem, cell_size)
hillshade = surtgis.hillshade_compute(
    dem, cell_size, azimuth=315.0, altitude=45.0)

# Florinsky curvatures
mean_curv = surtgis.advanced_curvature(
    dem, cell_size, ctype="mean_h")
gaussian_curv = surtgis.advanced_curvature(
    dem, cell_size, ctype="gaussian_k")

# Hydrology pipeline
filled = surtgis.priority_flood_fill(dem, cell_size)
twi = surtgis.twi_compute(dem, cell_size)
hand = surtgis.hand_compute(dem, cell_size)
streams = surtgis.stream_network_compute(
    dem, cell_size, threshold=100.0)
```

The same algorithms are available in the browser through the WebAssembly bindings, enabling interactive web applications where users can upload a DEM and compute terrain derivatives client-side without transmitting data to a server.

Cloud-native end-to-end workflow. A second representative usage scenario is the cloud-native pipeline that exercises the `surtgis-cloud` crate together with the algorithms layer. From the command line, three invocations take the user from a STAC catalogue query to a derived spectral index, with no

intermediate Python or shell glue (Figure 2): (i) `surtgis stac composite` searches a STAC endpoint (Microsoft Planetary Computer or Earth Search), downloads the matching scenes via HTTP range reads against the source COG assets, applies the catalogue's cloud-mask convention (SCL for Sentinel-2 L2A or QA_PIXEL for Landsat) to each scene, and reduces the temporal stack to a per-band median composite written as GeoTIFF; (ii) `surtgis imagery ndvi` computes the normalized difference vegetation index from the red and near-infrared composite bands; (iii) `surtgis info` prints summary statistics for verification. On the test hardware, this end-to-end pipeline runs in approximately 65 s for a 5×5 km bounding box (one month of Sentinel-2 imagery, three cloud-free scenes, two output bands) over a typical residential connection, and the full pipeline is reproducible from a single command-line block included in the project README.

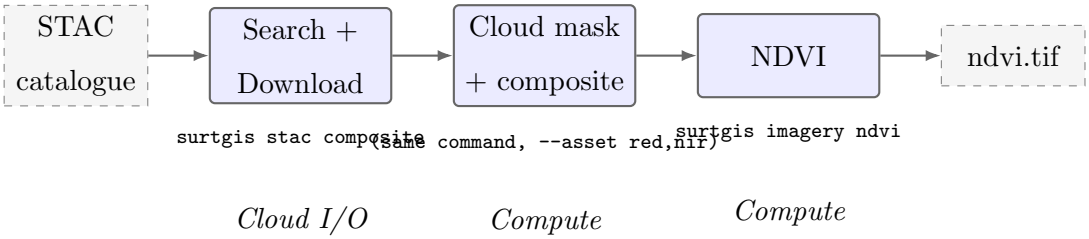


Figure 2: Cloud-native usage scenario: from a STAC catalogue query to a derived spectral index in three command-line invocations. The first `stac composite` call performs HTTP range reads against the source COG assets, applies the catalogue's cloud-mask convention (SCL for Sentinel-2 L2A or QA_PIXEL for Landsat), and reduces the temporal stack to a per-band median composite. The second `imagery ndvi` call combines the red and NIR composite bands. End-to-end runtime is approximately 65 s on a 5×5 km bounding box over a residential connection, with no intermediate Python glue or shell pipelines.

4. Application example: Andean terrain characterization

To illustrate SurtGIS's workflow capabilities, we present a terrain characterization of a mountainous region in the central Chilean Andes (approximately 33.5°S, 70.3°W), using a Copernicus DEM 30 m tile reprojected to UTM zone 19S (662 × 785 cells, cell size = 28.4 m). The study area spans elevations from 2,859 m to 5,979 m and covers approximately 19 × 22 km of rugged terrain characterized by deeply incised valleys and steep ridge systems. We chose this high-relief Andean setting because it exercises SurtGIS's full range of terrain and hydrological algorithms—steep slopes, high curvature contrasts, deep depressions, and a well-defined drainage network—rather than for operational flood mapping, which would require flatter, lower-elevation terrain and observed inundation data for calibration. This type of multi-layer terrain characterization is commonly required for landslide susceptibility mapping, hydrological modeling, and habitat characterization (Amatulli et al., 2018).

SurtGIS computes eight analysis layers from the DEM in a single Python session (Figure 3): slope, aspect, hillshade, mean curvature (H), geomorphon classification, depression-filled surface, Topographic Wetness Index (TWI), and Height Above Nearest Drainage (HAND) with stream network extraction. The entire workflow—including GeoTIFF I/O—executes in 0.3 s on the test hardware (0.5 M cells; for performance on larger DEMs, see Section 5).

Terrain morphology. The mean slope of 29° confirms the rugged character of the study area (Figure 3b). Florinsky's mean curvature H (Figure 3c) reveals the ridge-valley structure: positive values (red) mark convex ridges and spurs where erosion dominates, while negative values (blue) identify concave

hollows and channels where deposition occurs. The curvature field provides information beyond slope alone, as it distinguishes planar slopes from convergent and divergent landforms. Access to the full 14-curvature system enables further characterization: horizontal curvature k_h identifies zones of flow convergence (concavity in the plan view) relevant to landslide initiation, while the difference curvature E quantifies local flow asymmetry between principal directions. These additional curvatures are computed in the same Evans–Young derivative pass as H and K, adding negligible overhead to the workflow.

Landform classification. The geomorphon algorithm (Jasiewicz and Stepinski, 2013), applied with a lookup radius of 10 cells (∼284 m) and a flatness threshold of 1°, classifies the study area into three dominant landform types (Figure 3d): slope (44.9%), hollow (27.1%), and spur (26.5%), with minor contributions from peaks (0.9%) and valleys (0.2%). The near-absence of flat terrain is consistent with the high-gradient Andean environment. These proportions are sensitive to the lookup radius parameter: smaller radii detect finer-scale features (increasing the proportion of slope-class cells), while larger radii generalize the classification toward broader landform categories (Jasiewicz and Stepinski, 2013). This automated classification provides a parameter-parsimonious alternative to manual terrain unit delineation.

Hydrological analysis. The TWI (Kopecký et al., 2021) identifies zones of potential soil saturation in valley bottoms (Figure 3e), while HAND values (Nobre et al., 2011) delineate the vertical distance to the nearest drainage channel (Figure 3f). The extracted stream network (blue overlay in Figure 3f, ∼32,000 stream cells) follows the dendritic drainage pattern typical

of this geological setting. HAND values reach 1,698 m on the highest ridges, providing a continuous indicator of flood susceptibility that is increasingly used in operational flood mapping.

Flood susceptibility mapping. To demonstrate an applied environmental analysis, we use HAND thresholds from Nobre et al. (2011) (calibrated for Amazonian floodplains; illustrative for this Andean setting) to delineate flood susceptibility zones (Figure 4): high risk (HAND < 5 m, 3.4%), moderate (5–10 m, 1.7%), low (10–15 m, 1.5%), minimal (15–25 m, 2.7%), and negligible (> 25 m, 85.9%); the remaining $\sim$4.8% corresponds to NoData boundary cells. The combined high-to-moderate risk zone covers 5.1% of the study area, concentrated along valley bottoms. We note that HAND values are sensitive to the stream extraction threshold (100 cells, $\sim$0.08 km^2): varying between 50–200 cells shifts the HAND distribution, though the large-scale spatial pattern is preserved. These results are presented as a workflow demonstration, not an operational flood hazard map.

Integration. The derived layers form a feature stack suitable for statistical modeling: slope, curvatures, and TWI are standard predictors in logistic regression or random forest models for landslide susceptibility (Amatulli et al., 2018). The key practical advantage is that SurtGIS computes all required layers from a single DEM in a unified workflow, eliminating the need to chain GDAL (for basic derivatives), GRASS (for hydrology), and manual scripting to integrate outputs with potentially incompatible data formats. For instance, a multi-hazard assessment on a 10K^2 DEM (100 M cells) requiring slope, curvatures, TWI, and HAND would complete in under 60 s with SurtGIS, compared to several minutes when orchestrating separate GDAL

and GRASS invocations with intermediate file transfers. The unified API also ensures spatial metadata consistency across all derived layers, avoiding the CRS and extent mismatches that frequently arise when chaining heterogeneous tools. This section demonstrates SurtGIS as a workflow tool; validation of specific environmental applications (e.g., calibrated flood mapping, landslide susceptibility modeling) against field observations is deferred to domain-specific studies.

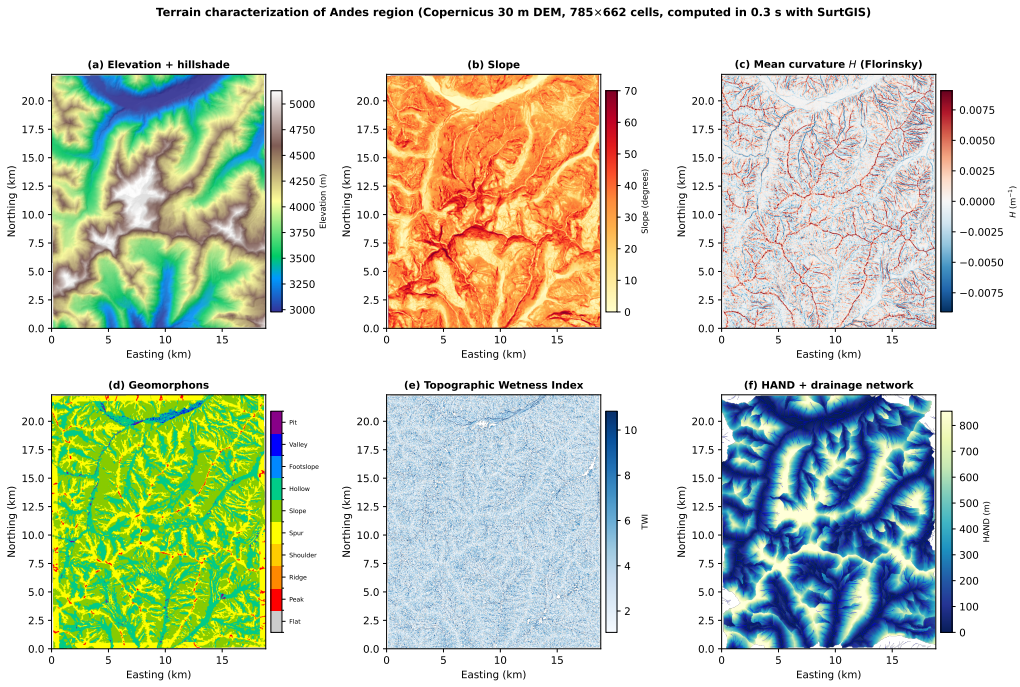


Figure 3: Terrain characterization of the central Chilean Andes (Copernicus 30 m DEM, 662×785 cells). (a) Elevation with hillshade overlay; (b) slope in degrees; (c) Florinsky mean curvature H (red = convex, blue = concave); (d) geomorphon landform classification; (e) Topographic Wetness Index; (f) Height Above Nearest Drainage with extracted stream network (blue).

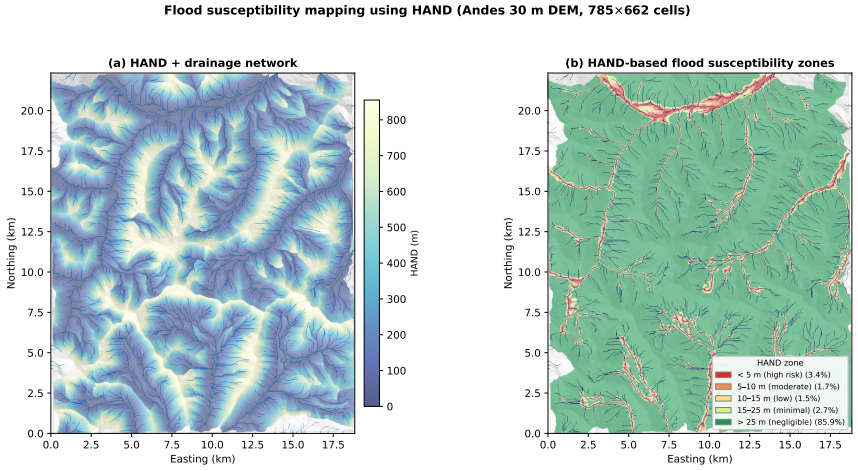


Figure 4: HAND-based flood susceptibility mapping. (a) Continuous HAND values with drainage network overlay; (b) classified flood susceptibility zones. The high-risk zone (HAND < 5 m, red) covers 3.4% of the study area, concentrated along the valley network.

5. Performance evaluation

We evaluate SurtGIS performance through four experiments: (1) a scalability benchmark comparing execution time against established tools across DEM sizes; (2) a numerical accuracy validation against analytical and cross-tool references; (3) a cross-platform comparison between native Rust, Python bindings, and single-threaded execution; and (4) a parallel-scaling analysis quantifying speedup, parallel efficiency, and the regimes that limit each algorithm class.

All benchmarks were executed on a single machine: Intel Core i7-1270P (4 performance cores up to 4.8 GHz + 8 efficiency cores up to 3.5 GHz, 16 threads), 38.8 GB DDR4 RAM, Ubuntu 24.04 LTS (kernel 6.14.0). The CPU governor was set to `powersave` (the system default, which allows the kernel to dynamically scale frequency); Intel Turbo Boost was enabled. We retained

the default governor rather than `performance` because: (a) it reflects the out-of-the-box configuration users are most likely to encounter, and (b) all tools are benchmarked under the same conditions, so the relative speedups remain valid even if absolute times vary with governor policy. Tool versions: GDAL 3.11.4, GRASS GIS 8.3.2, WhiteboxTools 2.4.0, SurtGIS 0.1.1 (commit `7493bc1`). The benchmark automation script (`benchmarks/run_benchmarks.py`), the native Rust benchmark (`bench_comparison.rs`), the WASM benchmark (`bench_wasm.mjs`), and all raw CSV results are included as supplementary material. DEM generation uses NumPy 2.4 with a fixed random seed (`seed=42`) for full reproducibility. The load-balancing experiment of Section 5.4.1 and the cross-platform MPI check of Section 5.5 additionally use a secondary platform with homogeneous cores and stable clocks—a Hetzner CCX33 instance (8 dedicated AMD EPYC-Milan cores at 2.0 GHz, 32 GB RAM, Ubuntu 24.04)—chosen because its uniform cores isolate load-balancing behaviour from the hybrid-core and thermal-throttling effects of the primary machine.

Versioning. Experiments 1–3 were performed on SurtGIS 0.1.1 (commit `7493bc1`) and reported here unchanged. The parallel-scaling analysis (Experiment 4, Section 5.4) was performed on the current development tree, released as v0.7.0. The 127 algorithms benchmarked are unchanged across these versions: the v0.6.x and v0.7.0 releases added cloud-native I/O (STAC and COG readers), out-of-core streaming variants for twelve terrain algorithms (Section 2.2), Python and CLI tooling for CNN patch extraction, and a documentation site, none of which modify the per-cell algorithm or alter the parallel-decomposition strategies analysed in this paper. To verify continuity we re-ran the Experiment 1 SurtGIS measurements on v0.7.0 ($n = 10$ per

cell, same DEMs, same harness): v0.7.0 produces equal or faster full-pipeline times across all six algorithms at both $5K^2$ and $10K^2$, with reductions of 4–61 %. The speedup ratios reported in Tables 6–7 are therefore conservative on the current release. The pre-1.0 development policy is to pre-announce breaking API changes one release ahead under a *Breaking* heading in the changelog; no breaking change has occurred between v0.1.1 and v0.7.0 affecting the algorithms benchmarked here.

The exact command lines used for each tool are:

- **SurtGIS**: `cargo run --example bench_comparison --release -- --size N --algo slope`
- **GDAL**: `gdaldem slope input.tif output.tif -of GTiff -b 1` (default Horn algorithm, `-compute_edges` not used to match SurtGIS boundary handling)
- **GRASS**: `grass --tmp-location EPSG:32719 --exec bash -c "r.in.gdal input=... output=dem; r.slope.aspect elevation=dem slope=out; r.out.gdal input=out output=..."`
- **WBT**: `whitebox_tools -r=Slope -v --dem=input.tif -o=output.tif`

5.1. Experiment 1: Scalability

We compared wall-clock execution time for six algorithms against GDAL 3.11, GRASS GIS 8.3, and WhiteboxTools 2.4 on synthetic DEMs of increasing size: $1K^2$ (1 M cells), $5K^2$ (25 M), $10K^2$ (100 M), and $20K^2$ (400 M cells). Synthetic DEMs were generated using a fractal Brownian surface (Hurst ex-

ponent = 0.7, fixed seed for reproducibility) to ensure identical input data across all tools.

The algorithms tested are: (a) slope, (b) aspect, (c) hillshade, (d) TPI, (e) Priority-Flood depression filling, and (f) D8 flow accumulation. These cover both simple moving-window operations (a–d) and graph-based algorithms with different computational complexity (e–f).

Each measurement was repeated 10 times after 3 warm-up runs. We report median execution time; interquartile range (IQR) is shown in the scalability figures and reported where relevant. The coefficient of variation (CV) was below 10% for all reported measurements. Across DEM sizes, per-algorithm median CV ranged from 0.8% (fill at $1\,\mathrm{K}^2$) to 4.1% (slope at $20\,\mathrm{K}^2$); the single highest individual CV was 6.4% (slope at $20\mathrm{K}^2$, one run). Given the sample sizes ($n = 10$ for scalability experiments, $n = 3$ for thread scaling), speedup differences smaller than $\sim$20% should be interpreted with caution. Since speedup is the ratio of two medians, its uncertainty depends on the variability of both tools; with CV $< 5\%$ for most measurements, the headline speedup ratios (e.g., slope vs GDAL 1.8×) are stable within ± 0.1–$0.2\times$ across the IQR.

Threading configurations. Because the tools differ in their parallelism support, we state each configuration explicitly rather than relying on defaults. SurtGIS uses Rayon's default thread pool (16 threads on the test machine); we additionally report SurtGIS *single-threaded* (`RAYON_NUM_THREADS=1`) so that a one-core, like-for-like comparison against single-threaded tools is available throughout. GDAL's `gdaldem` slope/aspect/hillshade kernels are single-threaded by design—GDAL exposes no threading option for these operations

(`GDAL_NUM_THREADS` governs warping and compression, not `gdaldem`)—so its timings reflect the tool's only available configuration, not a handicap we imposed. GRASS GIS `r.slope.aspect` supports OpenMP parallelism via `nprocs`; we report GRASS at both `nprocs=1` and `nprocs=16`. WhiteboxTools parallelises by default across all cores. To make the effect of these configurations explicit, Table 5 reports all configurations side by side; the headline cross-tool speedups elsewhere in this section use each tool's fastest available configuration (multi-threaded where supported). The comparison thus includes two further multi-threaded geocomputation frameworks beyond SurtGIS—WhiteboxTools (shared-memory) throughout, and TauDEM (MPI) for the hydrological algorithms (Section 5.5).

Table 5: Full-pipeline wall-clock time (s, median of three runs) under explicit threading configurations, for the three focal kernels on $10K^2$ (100 M cells) and $20K^2$ (400 M cells) DEMs. "s-1t/s-16t" = SurtGIS single-/16-thread; "GDAL" is single-threaded by design; "GR-1t/GR-16t" = GRASS `r.slope.aspect` at `nprocs` 1/16 (no `nprocs` for `r.relief`, hence n/a for hillshade); "WBT" = WhiteboxTools (multi-thread default). GRASS pipeline times include its per-call import/region/export overhead.

Algorithm	DEM	s-1t	s-16t	GDAL	GR-1t	GR-16t	WBT
Slope	$10K^2$	3.82	2.30	3.78	42.36	52.40	7.04
	$20K^2$	15.51	13.96	15.52	115.24	34.16	32.63
Aspect	$10K^2$	5.27	2.51	5.04	10.42	6.77	6.97
	$20K^2$	20.24	18.89	20.70	38.88	30.93	28.27
Hillshade	$10K^2$	9.79	3.85	2.43	20.64	n/a	7.53
	$20K^2$	51.88	34.89	12.38	132.32	n/a	36.91

Effect of competitor parallelism. Enabling GRASS's `nprocs=16` confirms that parallel configuration matters: it reduces GRASS slope time at $20K^2$ from 115 s to 34 s (3.4×), though at $10K^2$ the per-call import/export overhead dominates and `nprocs` yields no benefit (and is marginally slower). Because the GRASS pipeline times include format import and export that the in-process tools do not pay, they are an upper bound on its algorithmic cost; the comparison is therefore conservative in GRASS's favour, and the figures here are whole-tool, not kernel-only, ratios. Even in its fastest multi-threaded configuration, GRASS remains slower than 16-thread SurtGIS by 2.4–15× across the focal kernels, and WhiteboxTools—a parallel-by-default framework—by 1.9–3.1×. The single exception remains hillshade, where single-threaded GDAL is fastest (its SIMD-optimised trigonometry outweighs the threading gap), consistent with Section 5.2; 16-thread SurtGIS hillshade nonetheless overtakes its own single-thread time by 1.5–2.5× and beats both GRASS configurations and WhiteboxTools. The cross-tool ratios reported below therefore hold against multi-threaded, not artificially single-threaded, competitors.

To ensure a fair comparison, every tool—including SurtGIS—is measured on the same end-to-end pipeline: read a GeoTIFF from disk, compute the result, and write the output GeoTIFF to disk. SurtGIS uses its native Rust GeoTIFF reader/writer (based on the `tiff` crate), with no GDAL dependency. GDAL is invoked via its `gdaldem` CLI; WhiteboxTools via its binary; GRASS GIS via `grass --tmp-location`. All input DEMs are stored as uncompressed Float32 striped GeoTIFFs (no tiling, no compression) to eliminate codec-related variance and ensure the comparison reflects algorith-

mic performance rather than I/O library optimization. This is an important caveat: real-world DEMs are typically LZW- or DEFLATE-compressed, and SurtGIS's pure-Rust `tiff` crate is significantly slower than GDAL's optimized C codecs for decompression; the reported speedups should therefore be interpreted as upper bounds that may decrease in production workflows involving compressed data. We additionally report the SurtGIS compute-only time (excluding I/O) to quantify the algorithmic contribution.

5.1.1. Scalability results

Table 6 presents execution times for slope computation, the most commonly used terrain derivative. The "full pipeline" column includes GeoTIFF read/write, providing a direct comparison with other tools. Speedup factors are reported relative to SurtGIS full pipeline.

Table 7 summarizes all six algorithms at two representative scales: 5K^2 (25 M cells), the largest size for which all four tools complete basic terrain derivatives within the 300 s timeout, and 10K^2 (100 M cells), where SurtGIS completes all six algorithms but some tools exceed the timeout. The flow accumulation speedup at $10\,\text{K}^2$ is particularly notable: 23.1× over GRASS and 7.7× over WhiteboxTools.

Figure 5 shows execution time as a function of DEM size on log–log axes for slope, aspect, and hillshade. SurtGIS exhibits consistent scaling across all sizes, with speedup factors stable or increasing at larger scales.

Figure 6 presents the speedup heatmap across all algorithm–tool–size combinations. The highest speedups are observed for WhiteboxTools and GRASS at large DEM sizes, while GDAL exhibits more moderate overhead.

SurtGIS achieves the fastest full-pipeline time for slope and aspect across

Table 6: Execution time (seconds, median of 10 runs) for slope computation. "Compute" = algorithmic computation only; "Full" = including GeoTIFF I/O. Speedup relative to SurtGIS full pipeline.

DEM size	SurtGIS		GDAL 3.11	GRASS 8.3	WBT 2.4[b]
	Compute	Full			
1K × 1K	0.012	**0.020**	0.10 (5.1×)	0.62 (31.2×)	0.39
5K × 5K	0.24	**0.69**	1.17 (1.7×)	3.37 (4.9×)	6.11
10K × 10K	0.86	**2.03**	3.57 (1.8×)	9.70 (4.8×)	19.70
20K × 20K	3.89	**9.61**	17.66 (1.8×)	43.38 (4.5×)	107.90

[b]WhiteboxTools uses a 3rd-order Taylor polynomial on a 5 × 5 neighborhood (Florinsky, 2016) rather than Horn's 3 × 3 kernel. This different derivative method (RMSE = 6.53° vs. Horn-based tools; Table 9) makes WBT slope times not directly comparable; speedup ratios are therefore omitted.

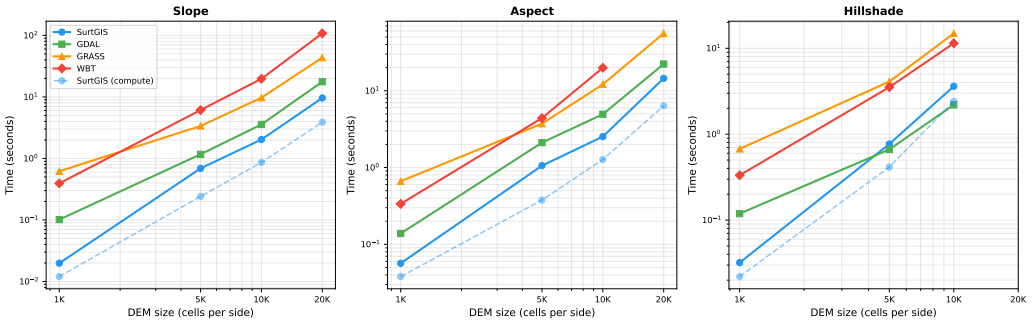


Figure 5: Log–log scalability of slope, aspect, and hillshade computation across four DEM sizes. Shaded bands indicate interquartile range across 10 runs.

Table 7: Execution time (seconds, median of 10 runs) for all algorithms at $5K^2$ and $10K^2$. "Compute" = computation only; "Full" = including GeoTIFF I/O. Speedup relative to SurtGIS full. T/O: timeout (300 s). Slope data repeated from Table 6 for cross-algorithm context; see that table for $1\,K^2$ and $20\,K^2$ results.

Scale	Algorithm	SurtGIS		GDAL 3.11	GRASS 8.3	WBT 2.4
		Compute	Full			
$5K^2$	Slope[b]	0.24	**0.69**	1.17 (1.7×)	3.37 (4.9×)	6.11
	Aspect	0.38	**1.06**	2.11 (2.0×)	3.72 (3.5×)	4.41 (4.2×)
	Hillshade	0.42	0.77	**0.67** (0.9×)	4.10 (5.3×)	3.53 (4.6×)
	TPI[a]	4.21	4.55	**0.55** (0.1×)	—	—
	Fill (PF)	4.44	**4.71**	—	T/O	12.17 (2.6×)
	D8 flow acc.	0.85	**1.12**	—	23.03 (20.6×)	8.36 (7.5×)
$10K^2$	Slope[b]	0.86	**2.03**	3.57 (1.8×)	9.70 (4.8×)	19.70
	Aspect	1.27	**2.53**	4.94 (2.0×)	12.11 (4.8×)	19.89 (7.9×)
	Hillshade	2.41	3.62	**2.20** (0.6×)	14.93 (4.1×)	11.46 (3.2×)
	TPI[a]	17.45	18.56	**2.09** (0.1×)	T/O	T/O
	Fill (PF)	27.82	**29.0**	—	T/O	46.33 (1.6×)
	D8 flow acc.	3.56	**4.68**	—	108.45 (23.1×)	36.23 (7.7×)

[a]GDAL computes TPI on a 3×3 neighborhood; SurtGIS uses a circular window with $r = 10$ cells, a fundamentally different and more computationally intensive operation (De Reu et al., 2013). These results are therefore not directly comparable. [b]WBT slope speedup omitted: WBT uses a different derivative method (see Table 6 note and Table 9).

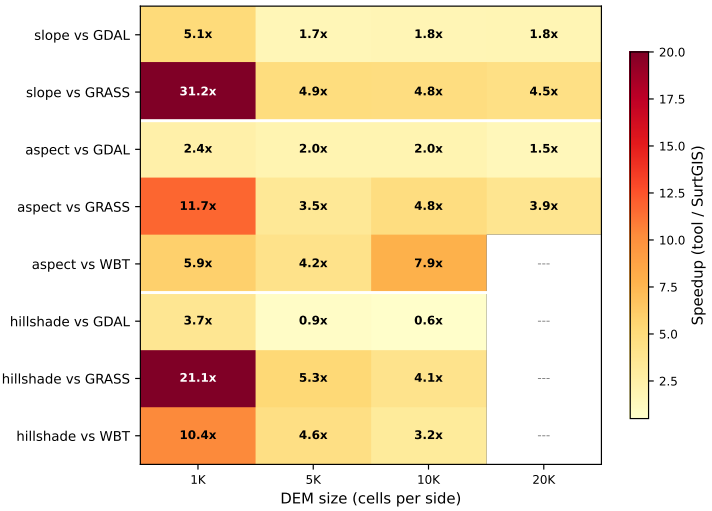


Figure 6: Speedup heatmap showing the ratio of competitor execution time to SurtGIS for slope, aspect, and hillshade across DEM sizes.

all tested sizes. For slope, speedups over GDAL range from 1.7–5.1×, stabilizing at 1.8× from 10 K^2 (100 M cells) onward. For aspect (Table 7), the speedup over GDAL is 2.0× at 5 K^2 and 10 K^2 but decreases to 1.5× at 20 K^2 (SurtGIS 14.5 s vs. GDAL 22.3 s; 20 K^2 data in supplementary CSV), likely reflecting GDAL's SIMD-optimized `atan2` in its aspect computation. Against GRASS and WhiteboxTools at 5 K^2–10 K^2, advantages are 3.5–4.8× over GRASS and 4.2–7.9× over WhiteboxTools. The advantage is most pronounced at the 1 K^2 scale, where SurtGIS's low-overhead native I/O and parallel computation dominate the fixed-cost overheads of external tools.

For hillshade, GDAL is marginally faster at 5 K^2 and above (0.9× at 5 K^2, 0.6× at 10 K^2), likely reflecting GDAL's SIMD-optimized trigonometric operations (`sin`/`cos`) in its C implementation. SurtGIS's hillshade compute time (2.41 s at 10 K^2) remains faster than GRASS (14.9 s) and WhiteboxTools

(11.5 s).

For hydrological algorithms, where computation dominates I/O, SurtGIS shows substantial advantages: 1.6–2.6× faster than WhiteboxTools for depression filling (decreasing at larger scales as the compute-bound portion dominates) and 7.5–20.6× faster for D8 flow accumulation. GRASS exceeds the 300 s timeout for fill at 5 K^2 and above.

The I/O breakdown (Table 7, Compute vs Full columns) reveals that file I/O accounts for 35–65% of SurtGIS's total time for simple derivatives, compared to <10% for computationally intensive algorithms such as fill and flow accumulation. SurtGIS's native Rust GeoTIFF reader/writer based on the `tiff` crate achieves competitive throughput for uncompressed data but does not implement the streaming I/O and SIMD codecs available in GDAL's C library.

I/O architecture. SurtGIS and WhiteboxTools load the entire raster into memory (read→compute→write), while GDAL and GRASS use streaming I/O that overlaps reading, computation, and writing. The "Compute" columns in Tables 6–7 isolate the algorithmic contribution. The in-memory model favors repeated analyses on the same dataset (as in the case study, Section 4), while streaming is preferable when dataset size approaches available RAM (Supplementary Table S2).

For TPI, GDAL computes a simple 3×3 neighborhood difference, while SurtGIS uses a circular window of radius 10 cells (∼314 cells per pixel); multi-scale TPI is preferred for geomorphological analysis (De Reu et al., 2013; Newman et al., 2018).

5.1.2. Memory usage

Peak memory usage was measured with `/usr/bin/time -v` on Linux at $10K^2$ resolution (Supplementary Table S2). SurtGIS's default in-memory model requires 2.6 GB peak RSS for slope (vs. 435 MB for GDAL's streaming I/O), reflecting a $\sim$6.8$\times$ inflation factor over the raw 382 MB Float32 input. This would limit the in-memory model to approximately $6{,}500^2$ cells on 8 GB systems and $9{,}200^2$ cells on 16 GB systems. The v0.7 release additionally includes out-of-core streaming variants for twelve terrain algorithms (slope, aspect, hillshade, TPI, TRI, roughness, classical curvatures, geomorphons, multi-scale curvature, sky-view factor, openness, MRVBF; see Section 2.2) that hold a single row strip in RAM at a time, lifting this size limit at a modest throughput cost.

5.1.3. Thread scaling

A dedicated parallel-scaling analysis covering five algorithms across six thread counts $p \in \{1, 2, 4, 8, 12, 16\}$ is presented as Experiment 4 (Section 5.4). That experiment isolates compute-only time, separates speedup regimes by algorithm class, and discusses parallel efficiency and load-balancing implications.

5.2. Experiment 2: Numerical accuracy

5.2.1. Analytical validation

To validate numerical correctness, we compared SurtGIS outputs against closed-form analytical solutions on a Gaussian hill surface:

$$z(x, y) = A \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \tag{5}$$

with $A = 500\,\mathrm{m}$, $\sigma = 2000\,\mathrm{m}$, on a 1000×1000 grid with 10 m cell spacing. The analytical slope, aspect, and curvatures were derived from the closed-form partial derivatives of Eq. 5. Boundary cells (1-pixel border) were excluded from comparison. Note that this smooth, infinitely differentiable surface represents a best-case scenario for finite-difference methods; real DEMs with sensor noise and terrain discontinuities will exhibit larger discretization errors. The cross-tool validation on a real DEM (Section 5.2) provides a complementary assessment under realistic conditions.

Table 8 presents the first-order results. SurtGIS achieves high numerical accuracy: slope RMSE of 5.7×10^{-5} degrees, aspect maximum angular error of 2.2×10^{-4} degrees, and $R^2 = 1.000000$ for both curvatures.

Table 8: SurtGIS accuracy against analytical solutions on a Gaussian hill (1000×1000 cells). Values rounded to significant digits. [a]Aspect R^2 is not reported because aspect is a circular variable; RMSE and MAE use angular distance.

Derivative	RMSE	MAE	R^2
Slope	5.7×10^{-5}°	4.0×10^{-5}°	1.000000
Aspect[a]	8.7×10^{-5}°	6.7×10^{-5}°	—
Mean curv. H	$< 10^{-6}\,\mathrm{m}^{-1}$	$< 10^{-6}\,\mathrm{m}^{-1}$	1.000000
Gaussian curv. K	$< 10^{-6}\,\mathrm{m}^{-2}$	$< 10^{-6}\,\mathrm{m}^{-2}$	1.000000

Validation of all 14 Florinsky curvatures on the same Gaussian hill (Supplementary Table S4) confirms RMSE below 4×10^{-9} for all types, with 11 of 14 yielding $R^2 = 1.000000$ (six decimal places). The three quantities that are analytically zero on radially symmetric surfaces (k_{ve}, K_r, rotor) show RMSE $< 10^{-9}$, confirming negligible absolute error. Eight Florinsky alge-

braic identities (Florinsky, 2025)—including $k_{\min} = H - M$, $k_{\max} = H + M$, $K = H^2 - M^2$—hold to within 1.6×10^{-8}.

Because the Gaussian hill is radially symmetric, three curvatures (k_{ve}, K_r, rotor) are identically zero and thus cannot be tested for proportional accuracy. To address this limitation, we repeated the validation on an anisotropic sinusoidal ridge $z = A\sin(k_x x)\cos(k_y y)$ with $A = 500\,\mathrm{m}$, $\lambda_x = 4000\,\mathrm{m}$, $\lambda_y = 3000\,\mathrm{m}$ on a 2000×2000 grid (5 m cells), where all 14 curvatures are non-trivial. Excluding the bottom 10% of gradient magnitude—a threshold chosen because gradient-dependent curvatures (k_h, k_v, k_{he}, k_{ve}, K_a, K_r, rotor) contain $\|\nabla z\|$ in their denominators and become numerically singular as $\|\nabla z\| \to 0$; results are stable for thresholds between 5% and 20%—all 14 curvatures achieve $R^2 = 1.000000$ with maximum RMSE $= 5.9 \times 10^{-8}$, confirming numerical correctness of the complete Florinsky system on non-symmetric terrain.

Cross-validation against SAGA GIS (Evans method; seven overlapping curvatures; Supplementary Table S5) revealed systematic methodological differences: the Laplacian agrees well (RMSE $= 2.5 \times 10^{-7}\,\mathrm{m}^{-1}$, $R^2 = 0.9999$), confirming correct second-derivative computation, while gradient-dependent curvatures (k_h, rotor) diverge due to different formulations of slope-normalized terms (Minár et al., 2020). These differences confirm that SurtGIS implements the Florinsky (2025) formulation rather than the SAGA variant.

5.2.2. Cross-tool agreement

We compared slope values computed by SurtGIS, GDAL, GRASS GIS, and WhiteboxTools on a real-world DEM (Copernicus 30 m, Andes region,

reprojected to UTM zone 19S, 662 × 785 cells, cell size = 28.4 m). Using a projected coordinate system ensures consistent metric units across all tools. Table 9 shows pairwise agreement.

Table 9: Pairwise slope agreement between tools on Andes 30 m DEM (UTM 19S). Horn-based tools (SurtGIS, GDAL, GRASS) show near-perfect agreement.

	RMSE (°)	R^2	% within 0.1°
SurtGIS vs GDAL	1.6×10^{-4}	1.000	100%
SurtGIS vs GRASS	1.0×10^{-6}	1.000	100%
GDAL vs GRASS	1.6×10^{-4}	1.000	100%
SurtGIS vs WBT	6.53	0.771	16.2%
GDAL vs WBT	6.53	0.771	16.2%
GRASS vs WBT	6.53	0.771	16.2%

SurtGIS, GDAL, and GRASS GIS implement Horn's 3rd-order finite difference method (Horn, 1981) and agree to within 1.6×10^{-4} degrees (RMSE), with SurtGIS and GRASS achieving near-identical results (RMSE $< 10^{-6}$). WhiteboxTools uses a different derivative estimation method, resulting in larger systematic differences (RMSE = 6.53°). When excluding pixels adjacent to nodata boundaries, 87.6% of cells agree within 1° and 98.8% within 2°, confirming that the discrepancy is methodological rather than indicative of errors.

5.2.3. Hydrological algorithm validation

Cross-tool comparison of depression filling, D8 flow accumulation, and stream network extraction on the Andes DEM (Supplementary Table S6,

Figure 7) confirms high agreement. Depression filling produces near-identical results across all three tools (RMSE < 0.15 m, $R^2 = 1.000$). For D8 flow accumulation, SurtGIS and WhiteboxTools show excellent agreement ($R^2 = 0.949$ on log-transformed values, Pearson $r = 0.987$); the GRASS comparison shows lower correlation ($R^2 = 0.875$) because GRASS `r.watershed` accumulates upstream area (m^2) rather than cell counts. Stream networks exhibit high overlap: SurtGIS vs. GRASS F1 = 0.973, SurtGIS vs. WBT F1 = 0.938.

Scope of hydrological validation. The benchmarks focus on Priority-Flood filling and D8 flow accumulation because these are the algorithms for which direct cross-tool comparison is straightforward. Pixel-by-pixel comparison of SurtGIS D-infinity flow direction against TauDEM 5.3 (Tarboton, 1997) on the Andes DEM ($N = 508{,}918$ valid cells) yields RMSE $< 10^{-4°}$ with 100% of cells within 1°, confirming near-exact agreement with the reference implementation. The Lindsay (2016) breach algorithm executes in 29.8 s on the Andes DEM (0.5 M cells, median of 5 runs) but was excluded from cross-tool comparison because WhiteboxTools's breach implementation differs in parameterization (constrained vs. unconstrained). TWI cross-tool comparison between SurtGIS (D8-based) and GRASS `r.topidx` (MFD-based) on the Andes DEM yields RMSE = 2.72 and Pearson $r = 0.44$, confirming the well-documented sensitivity of TWI to flow routing algorithm choice (Kopecký et al., 2021); users should select the flow direction method appropriate for their application.

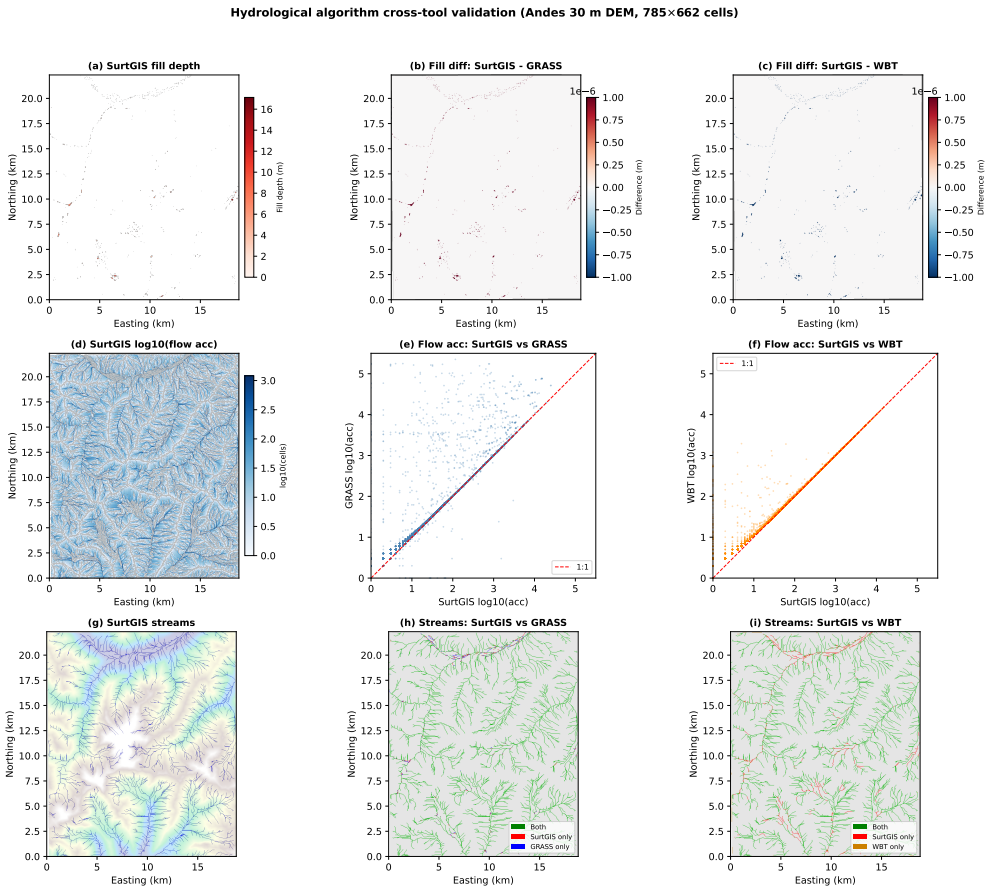


Figure 7: Hydrological algorithm cross-tool validation. Top row: depression filling differences. Middle row: D8 flow accumulation scatter plots (log scale). Bottom row: stream network overlap (green = agreement, red/blue = tool-specific differences).

5.3. Experiment 3: Cross-platform performance

We measured execution time across deployment targets for slope, aspect, and hillshade on a 5000×5000 DEM (25 M cells): (a) native Rust multi-threaded (Rayon, 16 threads), (b) native Rust single-threaded, (c) Python bindings via PyO3, and (d) WebAssembly executed in the V8 engine (Node.js 22.19, V8 12.4). The WASM binary (564 KB, compiled via `wasm-pack` with release optimizations) was benchmarked on the same hardware and DEM files as the native targets using Node.js's V8 engine. WASM execution is single-threaded, as SharedArrayBuffer-based multi-threading is not yet implemented. Note that browser-based WASM performance may differ from Node.js due to browser security mitigations (e.g., reduced timer resolution), garbage collection pauses, and JIT compilation differences; the Node.js measurements should be considered an upper bound on WASM performance.

Table 10: Cross-platform execution time (seconds, median of 10 runs) on a $5{,}000 \times 5{,}000$ DEM. Compute-only values may differ from Table 7 due to separate measurement harnesses and system load conditions.

Algorithm	Rust (MT)	Rust (ST)	Python	WASM (V8)
Slope	**0.19**	0.51 (2.6×)	0.50 (2.7×)	2.33 (12.3×)
Aspect	**0.16**	0.88 (5.7×)	0.61 (3.9×)	3.07 (19.2×)
Hillshade	**0.28**	2.21 (8.0×)	0.89 (3.2×)	8.88 (31.7×)

Values in parentheses: slowdown relative to Rust multi-threaded.

Table 11 presents additional WASM benchmarks at $1K^2$ (1 M cells), a size representative of browser-based interactive use cases. At this scale, all six algorithms complete within seconds, demonstrating the feasibility of real-

time terrain analysis in web applications.

Table 11: WASM execution time (seconds, median of 10 runs, V8 engine) on a 1,000×1,000 DEM. Comparison with native Rust compute-only time. Peak memory: 120–151 MB.

Algorithm	Rust (MT)	WASM (V8)	Slowdown
Slope	0.012	0.129	10.7×
Aspect	—	0.097	—
Hillshade	—	0.229	—
TPI (r=10)	—	2.61	—
Fill (PF)	—	0.398	—
D8 flow acc.	—	0.290	—

Figure 8 shows the grouped comparison. Multi-threaded Rayon execution provides 2.6–8.0× speedup over single-threaded execution, with hillshade benefiting most due to its higher per-cell computational cost. Python bindings add 2.7–3.9× overhead relative to native multi-threaded Rust, which includes NumPy↔Rust data marshalling and the Python GIL. Notably, the Python binding overhead is comparable to or less than the single-threaded Rust time, indicating that the PyO3 bridge successfully leverages multi-core parallelism even from Python.

WebAssembly execution on the V8 engine adds 10–32× overhead relative to multi-threaded native Rust, and 4.6–3.5× relative to single-threaded Rust for slope and aspect, respectively. The higher hillshade overhead (4.0× vs. single-threaded) reflects the cost of trigonometric operations (`sin`/`cos`) in WASM's software floating-point path versus native SIMD instructions. Despite this overhead, WASM enables terrain analysis directly in web browsers

without server-side processing—a capability unavailable in GDAL, GRASS, or WhiteboxTools. A live web demonstration with 33 algorithms is available at the project repository.

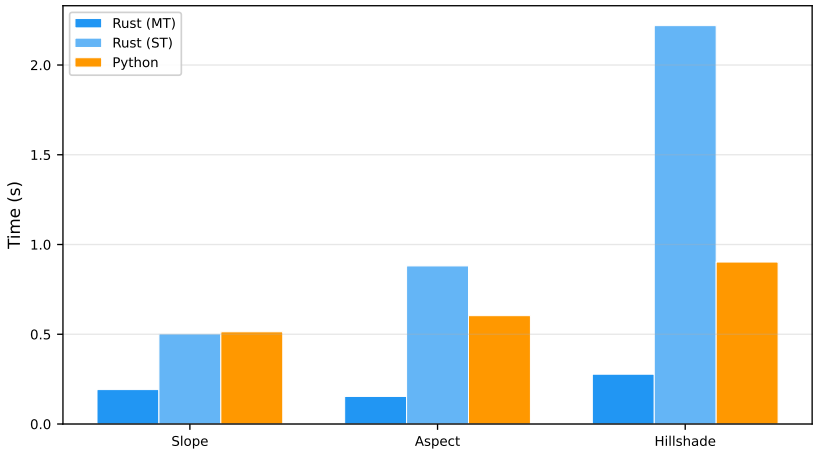


Figure 8: Cross-platform performance comparison for slope, aspect, and hillshade on a 5K^2 DEM. Overhead ratios relative to Rust MT annotated above bars. WASM bars extend beyond the figure range for hillshade (8.88 s).

In-browser WASM vs server-based (B/S) deployment. Beyond raw compute time, the deployment model itself differs between the two ways a user can run a SurtGIS computation from a browser. Table 12 contrasts the in-browser WebAssembly execution path with a classic Browser/Server (B/S) architecture (e.g. a JavaScript front-end such as Vue calling a Java/Spring Boot service that itself wraps GDAL or GRASS). The end-to-end user-perceived latency in the B/S case includes data upload, server queue and execution, and result download, which for a 4 MB compressed GeoTIFF over a typical 10 Mbps residential uplink is ∼3.2 s of network time alone, before any compute. The in-browser WASM path eliminates these phases entirely: the

user selects a file via the local `File` API and the result raster is materialised in browser memory without any HTTP round-trip. For DEMs that fit in browser memory ($\lesssim 2\text{K}^2$ in current Wasm builds, limited by the 32-bit linear memory addressing), this is decisive even though server-side compute on a dedicated CPU may be faster in absolute terms. The trade-offs invert at scale: B/S architectures amortise infrastructure across many tenants and can process DEMs too large for client RAM, but require server provisioning, authentication, queueing, and incur per-tenant compute cost.

A representative numeric comparison: a $1\,000 \times 1\,000$ slope computation in the browser takes 0.13 s on V8 (Table 11); the same operation on a typical t3.micro AWS instance running GDAL takes ∼0.10 s, but adds upload of a ∼4 MB GeoTIFF (∼3.2 s on a 10 Mbps uplink) and download of the slope output. End-to-end the in-browser path is therefore ∼25× faster from the user's perspective on this size class, in addition to the privacy and offline-capability benefits. The crossover point at which the B/S model wins on latency depends on uplink speed and server provisioning, but for the interactive sub-second use cases that motivate a browser GIS (visual exploration, parameter sweeps, teaching tools), the in-browser model is dominant on the inputs it can hold in memory.

5.4. Experiment 4: Parallel scaling analysis

The scalability experiment in Section 5.1 compares SurtGIS to other tools at default thread settings, but does not isolate how SurtGIS itself scales with thread count. We address this here with a dedicated experiment: speedup $S(p) = T(1)/T(p)$ and parallel efficiency $E(p) = S(p)/p$ as a function of the number of threads p, for five algorithms representative of distinct parallel-

Table 12: Qualitative comparison of in-browser WebAssembly execution (this work) and a typical Browser/Server architecture using a JavaScript front-end and a Java/Spring Boot or Python back-end that wraps a native GIS library. "N" denotes input raster size; "T_w" and "T_s" are WASM and server compute time respectively.

Phase / property	In-browser WASM (SurtGIS)	B/S (Vue + Spring Boot, etc.)
Data upload	0 (local `File` API)	N bytes uploaded (network-bound)
Server queue	0	variable (load- and tenant-dependent)
Compute time	T_w (single-threaded WASM)	T_s (server CPU, often multi-threaded)
Result download	0 (in-page `Blob`)	M bytes downloaded (network-bound)
End-to-end latency	T_w	T_s + upload + queue + download
Data residency	local (never leaves client)	on server (privacy + governance)
Max raster size	limited by browser memory ($\sim$2 K^2 in 32-bit Wasm)	limited by server RAM
Operational cost	per-user CPU (free for the operator)	infrastructure + bandwidth per tenant
Offline capability	yes (Progressive Web App)	no (network-dependent)

decomposition regimes.

Methodology. We measure compute-only time (load and write are excluded from $T(\cdot)$) for slope, aspect, hillshade, Priority-Flood depression filling, and D8 flow accumulation on 5K^2 (25 M cells) and 10K^2 (100 M cells) synthetic fBm DEMs (same generator as Experiment 1). Thread counts $p \in \{1, 2, 4, 8, 12, 16\}$ are imposed via Rayon's `ThreadPoolBuilder::num_threads(p).build_global()`. Because Rayon's global pool can be initialised only once per process, the benchmark binary is re-launched for each p. Each $(\text{algorithm}, \text{size}, p)$ cell is run with 2 warm-up iterations followed by 5 measured repetitions; we report the median compute time. The hardware is the same Intel i7-1270P configuration described in Section 5: a hybrid topology of 4 performance cores (4.8 GHz, 2 SMT threads each) and 8 efficiency cores (3.5 GHz, single-threaded), totalling 16 logical threads. The hybrid layout is non-ideal for clean speedup curves but is representative of contemporary consumer hardware; we discuss its impact below.

Results. Figure 9 reports speedup curves and Figure 10 the corresponding parallel efficiency. Table 13 summarises $S(p)$ and $E(p)$ at the maximum thread count $p = 16$.

Three scaling regimes. The data partition cleanly into three regimes that match well-established parallel-raster theory (Guan and Clarke, 2010; Miao et al., 2017):

1. *Memory-bandwidth-bound focal kernels.* Slope, aspect, and hillshade are embarrassingly parallel by row strip with a fixed-radius halo; in principle, $E(p) \approx 1$ should be achievable. In practice, $S(16)$ plateaus

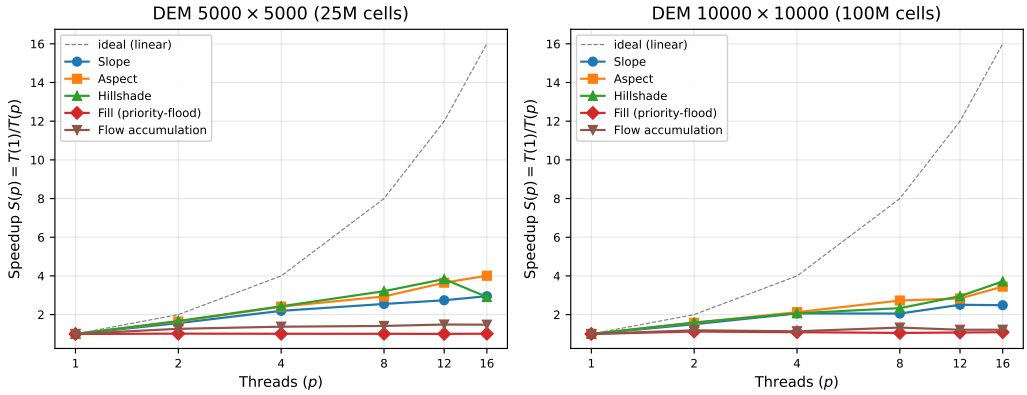


Figure 9: Parallel speedup $S(p) = T(1)/T(p)$ for five SurtGIS algorithms at $p \in \{1, 2, 4, 8, 12, 16\}$ threads, on 5K^2 (left) and 10K^2 (right) DEMs. Dashed line marks ideal linear speedup. Each point is the median of 5 measured runs after 2 warm-ups; compute-only time, I/O excluded.

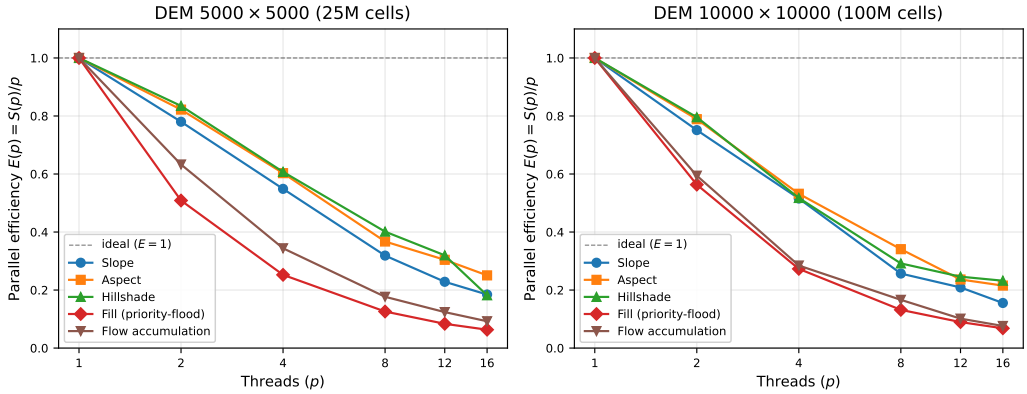


Figure 10: Parallel efficiency $E(p) = S(p)/p$ for the same runs as Figure 9. The horizontal dashed line at $E = 1$ is the ideal. Embarrassingly-parallel focal kernels (slope, aspect, hillshade) fall along a memory-bandwidth-bound trajectory; Priority-Flood and D8 accumulation drop sharply due to inherent sequential phases.

Table 13: Parallel speedup S and efficiency E at $p = 16$ threads on a 12-core / 16-thread Intel i7-1270P. $T(1)$ and $T(p)$ are median compute-only time over 5 reps with 2 warmup runs.

Algorithm	DEM size	$T(1)$ (ms)	$S(16)$	$E(16)$
Slope	5000×5000	392	2.95	0.18
Aspect	5000×5000	703	4.01	0.25
Hillshade	5000×5000	1354	2.91	0.18
Fill (priority-flood)	5000×5000	3230	1.01	0.06
Flow accumulation	5000×5000	1118	1.48	0.09
Slope	10000×10000	1591	2.49	0.16
Aspect	10000×10000	2783	3.45	0.22
Hillshade	10000×10000	5635	3.72	0.23
Fill (priority-flood)	10000×10000	21202	1.09	0.07
Flow accumulation	10000×10000	4370	1.22	0.08

between 2.5 and 4.0 on both DEM sizes. The ratio between observed and ideal scaling is consistent with the per-thread memory bandwidth requirement saturating the system's DDR4-3200 channel: at $f64$ precision, slope reads nine input cells per output cell ($\approx 72\,\mathrm{B}$ per cell), and at peak compute throughput each thread requires $\sim$1.5 GB/s, so 16 threads collectively demand $\sim$24 GB/s on top of write traffic, a substantial fraction of the $\sim$50 GB/s effective single-channel DDR4 bandwidth. Beyond 8 threads the curves flatten or, for hillshade at $5\mathrm{K}^2$, mildly regress (3.21$\times$ at p=12, 2.91$\times$ at p=16): efficiency-cores entering the pool contribute throughput at roughly half the per-thread rate

of performance-cores, while consuming a proportional share of the saturated memory bandwidth. This is fundamental to single-pass kernels on memory-constrained hardware rather than a limit of the SurtGIS implementation; on multi-channel server platforms (e.g. NUMA Xeon or EPYC), the same kernels would likely scale further before saturating.

2. *Amdahl-bound algorithms.* Priority-Flood depression filling (Barnes et al., 2014) relies on a single global priority queue: $S(16) \approx 1.05$ across both DEM sizes, indicating an effectively serial computation. Parallel variants exist—tile-decomposition schemes with boundary reconciliation (Barnes, 2017) are an obvious candidate—and integrating one is on the SurtGIS roadmap, but the present implementation prioritises algorithmic simplicity and exact result equivalence with the reference. The benchmark exposes this honestly: $E(p) \to 1/p$ is the expected behaviour of a sequential algorithm under thread sweeping and confirms there is no spurious parallelism inflating the reported $T(1)$.
3. *Dependency-aware partial parallelism.* D8 flow accumulation must respect the topological order of the flow DAG. SurtGIS's implementation processes cells in an order derived from the DAG's reverse-postorder, with limited per-stripe concurrency on independent sub-trees, yielding $S(16) \approx 1.2$–1.5. The scaling is sub-linear because most of the dependency structure on a single connected DEM lies on the critical path; a tile-and-reconcile strategy analogous to that of Barnes (2017) for fill would lift this ceiling, again without changing the per-cell algorithm.

Load balancing. Because Rayon uses work-stealing scheduling (Blumofe and Leiserson, 1999) and the row-strip decomposition assigns roughly equal cell

counts per worker, the three focal-kernel curves track each other within experimental noise across p. The departure from ideal E visible in Figure 10 is therefore overwhelmingly attributable to memory-bandwidth saturation and hybrid-core heterogeneity rather than to load imbalance per se: a perfect-balance run on the same hardware would still hit the same bandwidth ceiling. The Amdahl-bound regime (Priority-Flood) is also a load-balance question only in a degenerate sense, because the work itself is sequential.

Implications for users. The headline scaling claims in Section 5.1 (e.g. 1.7–2.0× over GDAL on slope) are obtained at default Rayon thread settings on the same hardware, so they implicitly include the bandwidth ceiling characterised here. Users running SurtGIS on multi-channel server platforms can therefore expect equal or better speedups than reported, while users restricted to a small number of cores (e.g. a 4-core laptop) can read off the expected slowdown directly from Table 13: $T(p)$ for $p = 4$ on 10K^2 slope is 771 ms, well within interactive use.

5.4.1. Cross-platform load balancing under spatial heterogeneity

The scaling analysis above uses dense synthetic DEMs, for which the row-strip partition is intrinsically balanced, and runs on the primary (hybrid-core) machine, whose memory-bandwidth ceiling can mask load-balance effects. To isolate the claim of Section 2.2—that dynamic work-stealing absorbs spatial workload heterogeneity without complexity-driven static partitioning—we run a focused experiment on a secondary platform with homogeneous cores and stable clocks: a Hetzner CCX33 instance (8 dedicated AMD EPYC-Milan cores at 2.0 GHz, 32 GB RAM, Ubuntu 24.04), benchmarking the

published SurtGIS release (v0.15.4, installed from crates.io). The adverse input for a static row-strip decomposition is a 10K^2 DEM masked to a rotated rectangle so that 29.2% of cells are NoData, concentrated in corner triangles—the pattern produced by WGS84→UTM reprojection, in which entire strips near the top and bottom are dominated by short-circuiting NoData cells while interior strips carry full work. If load balance depended on the static partition, the speedup curve on the masked DEM would fall well below that of a dense DEM of equal dimensions; if the work-stealing scheduler rebalances dynamically, the two curves should nearly coincide.

Table 14: Speedup $S(p) = T(1)/T(p)$ on a dense 10K^2 DEM versus a masked copy with 29.2% corner NoData (the reprojection-corner pattern, worst case for a static row-strip partition), on 8 homogeneous AMD EPYC cores. Median of five runs; median CV 0.8%. The masked curve tracking the dense curve indicates that work-stealing absorbs the spatial heterogeneity at runtime.

Algorithm	DEM	$T(1)$ (s)	$S(2)$	$S(4)$	$S(8)$	retention
Slope	dense	2.89	1.53	2.08	2.43	—
	masked	2.45	1.38	1.86	2.06	85%
Hillshade	dense	7.27	1.76	2.94	3.76	—
	masked	5.56	1.71	2.66	3.31	88%

Table 14 reports the result: the masked DEM retains 85% (slope) and 88% (hillshade) of the dense DEM's 8-core speedup. A static partition that left corner strips idle would scale markedly worse than this; the modest residual is consistent with the masked workload's reduced compute-to-overhead ratio—29% fewer kernel evaluations against an unchanged output-grid traversal and

I/O—rather than with load imbalance. This is the empirical complement to the design argument of Section 2.2: in a shared-memory runtime the load-balancing role played by coding aggregation and complexity-driven partition sizing in distributed-memory libraries (Qin et al., 2014; Wang et al., 2021) is discharged by the scheduler, so static, complexity-aware partitioning of the input is unnecessary for the within-node workloads that dominate terrain analysis. (This experiment also serves as a cross-platform check: the homogeneous-core results corroborate the bandwidth-bound scaling regime characterised on the primary machine in Section 5.4.)

5.5. Experiment 5: Comparison against an MPI framework

Experiments 1 and 4 compare SurtGIS to shared-memory tools. To position SurtGIS against the distributed-memory paradigm directly, we benchmark it against TauDEM (Tarboton, 2015), a mature MPI-based parallel geocomputation framework that implements the same flow-routing algorithms (D8 and D-infinity flow direction and accumulation), with comparable depression-filling functionality. Both tools run on the identical single node (Intel i7-1270P, 16 hardware threads); TauDEM is launched under `mpiexec` with $W \in \{1, 2, 4, 8, 16\}$ processes and SurtGIS with the matching `RAYON_NUM_THREADS`$= W$, on the same DEMs. We report this single-node comparison with an explicit caveat: it cannot exhibit TauDEM's principal design advantage, scaling *across* machines, which is irrelevant to the desktop, laptop, browser, and single-VM deployment surfaces SurtGIS targets but central to cluster workloads.

Protocol. A methodological asymmetry must be controlled. TauDEM's `pitremove` fills depressions to flat surfaces, which its flow-direction stage then resolves through an iterative flat-routing pass; SurtGIS's Priority-Flood fill imposes an ε drainage gradient and leaves no flats. On the $5K^2$ DEM, TauDEM's D8 flow-direction stage spent 5,576 s of a 5,596 s run resolving flats—99.6% of the time—whereas the same stage on an ε-filled input completed in 10.2 s (545× faster). Comparing the two fill semantics directly would therefore benchmark flat-resolution policy rather than parallel-computation efficiency. We instead adopt a stage-wise protocol: the fill stage is timed per tool on the raw DEM (semantics differ, reported separately), while flow direction and accumulation are timed for both tools on a *single shared* ε-filled DEM, so each stage compares identical computational work. The flat-routing observation is itself reported as a finding. Feeding TauDEM its own `pitremove` output instead of the shared ε-filled DEM would only *increase* its flow-direction time (the uncorrected, flats-included pipeline timing is retained in the supplementary material), so the shared-input protocol is conservative toward SurtGIS; the ε-gradient fill that leaves no flats is a SurtGIS design choice, not a cost removed from the comparison.

Results. SurtGIS completes the hydrological pipeline 2.6–5.5× faster than TauDEM at every matched worker count (Table 15). Two patterns are notable. First, neither tool's pipeline accelerates with more workers on a single node: the depression-fill stage is Amdahl-bound (Section 5.4) and file I/O is serial, so wallclock is dominated by components that within-node parallelism cannot reduce—consistent with the scaling regimes characterised in Experiment 4. Second, TauDEM degrades beyond the physical-core count: at

Table 15: SurtGIS (Rayon, shared-memory) versus TauDEM (MPI) on the hydrological pipeline (fill + flow direction + flow accumulation), identical hardware and DEMs, W matched workers. Wallclock seconds, median of three runs after one warmup. "T/S" is the TauDEM-to-SurtGIS ratio (higher = SurtGIS faster). Flow direction and accumulation operate on a shared ε-filled input (see text).

DEM	Method	W	SurtGIS (s)	TauDEM (s)	T/S
$5K^2$	D8	1	6.19	26.80	4.3×
		4	6.00	21.46	3.6×
		16	5.94	32.40	5.5×
	D-infinity	1	11.79	48.55	4.1×
		4	11.87	33.70	2.8×
		16	12.04	46.85	3.9×
$10K^2$	D8	1	31.11	119.77	3.8×
		4	29.05	90.98	3.1×
		16	34.17	126.48	3.7×
	D-infinity	1	54.20	207.97	3.8×
		4	52.74	136.92	2.6×
		16	60.74	179.78	3.0×

$W = 16$ on the 5K^2 D8 pipeline it is slower (32.4 s) than at $W = 4$ (21.5 s), reflecting MPI process-oversubscription overhead on 12 physical cores, whereas SurtGIS's work-stealing runtime is flat across worker counts. The result is consistent with the design argument of Section 2.2: for the data volumes that fit on a single node, a shared-memory model with zero inter-process communication avoids the message-passing and process-management overheads that an MPI runtime pays regardless of whether it is deployed on one node or many. MPI's advantage—and the rationale for the static decomposition strategies of Qin et al. (2014) and Wang et al. (2021)—emerges only when the problem outgrows a single machine, a regime we identify as future work rather than displace (Section 7). More generally, the crossover this comparison localises—shared-memory, zero inter-process-communication execution versus distributed-memory partitioning—is governed by problem size relative to a single node's capacity rather than by SurtGIS specifically: any single-node geocomputation library occupies the same design point for data volumes that fit one machine's memory, and the contribution of this experiment is to bound where that crossover lies on current commodity hardware. Run-to-run variation was consistent with the coefficients of variation reported for Experiment 1 (below ∼5% for most measurements), with larger scatter only in the process-oversubscription regime ($W = 16$ on 12 physical cores); ratios differing by less than ∼20% should be read with that caution.

Cross-platform robustness. We repeated this comparison on the homogeneous-core secondary platform of Section 5.4.1 (8 dedicated AMD EPYC cores), using the published SurtGIS release. SurtGIS remains faster than TauDEM at every matched worker count there as well, by 3.4–4.1× single-threaded

and 1.3–1.9× at 4–8 workers. The narrower multi-worker margin is informative: on homogeneous cores TauDEM's MPI scaling improves (its D8 $10K^2$ pipeline drops from 101 s at W=1 to 39 s at W=4), so the gap reflects genuine per-runtime efficiency rather than a hardware artifact of the primary machine, and SurtGIS retains the advantage on both platforms.

6. Comparison with existing tools

Table 16 compares SurtGIS with widely used geospatial analysis tools across key dimensions relevant to environmental modeling.

SurtGIS's primary differentiator is the combination of comprehensive algorithms, cross-platform deployment including WebAssembly, and fully open-source availability. While WhiteboxTools offers a comparable algorithm set, its advanced extensions are proprietary and it cannot compile to WebAssembly. GRASS GIS and SAGA GIS are powerful desktop/server tools but lack browser deployment and require complex installation procedures.

In terms of performance, the scalability benchmarks (Section 5.1) demonstrate that SurtGIS is consistently faster than GDAL for slope (1.8× at 400 M cells) and aspect (2.0× at 100 M cells), while GDAL is faster for hillshade due to SIMD-optimized trigonometry. SurtGIS is 4.5× faster than GRASS for slope at the 400 M cell scale, and 4.2–7.9× faster than WhiteboxTools for aspect (the WBT slope comparison is excluded due to different derivative methods; see Table 9). For hydrological algorithms, SurtGIS achieves 1.6–23.1× speedups over competitors. Importantly, all benchmarks measure the complete GeoTIFF pipeline (read, compute, write), not just in-memory computation.

Table 16: Feature comparison of geospatial terrain analysis tools.

Feature	SurtGIS	WBT[d]	GRASS	SAGA[e]	GDAL
Language	Rust	Rust[f]	C	C++	C++
Open source	Full	Partial[a]	Full	Full	Full
Browser (Wasm)	Yes	No	No	No	No
Python bindings	Yes	Yes	Yes	Yes	Yes
Florinsky 14 curv.	Yes	No	No	No	No
Flow directions	4[b]	3	3	3	1
Depression handling	3[c]	3	1	2	0
Parallel execution	Rayon (auto)	Multi-thread	OpenMP[g]	OpenMP	I/O threads

[a]WhiteboxTools Open Core is open source; advanced extensions require license. [b]D8, MFD, D-infinity, TFGA. [c]Planchon–Darboux, Priority-Flood, breach. [d]Lindsay (2016b). [e]Conrad et al. (2015); excluded from timing benchmarks because SAGA's CLI startup overhead (module loading, GDAL linkage) confounds single-algorithm timings, and its Horn-based terrain derivatives would fall between the GDAL and GRASS results already reported. [f]WhiteboxTools Open Core is written in Rust; earlier versions (pre-2018) were implemented in Go. [g]GRASS r.watershed uses OpenMP; most other modules are single-threaded.

Installation footprint. Beyond runtime performance, installation friction is a practical determinant of tool adoption, particularly in containerised scientific-computing environments where image size and cold-start time matter. We measured cold-start installation in fresh `ubuntu:24.04` Docker containers from base image to working CLI invocation; results are summarised in Table 17. SurtGIS via the prebuilt Linux release binary has the smallest on-disk footprint (22 MB) and competitive wall time (15 s end-to-end), no transitive system-library dependencies (no `apt` packages installed), and a two-command install (`curl` + `tar`). GDAL, GRASS, and WhiteboxTools each pull in 100–250 transitive Debian packages and 26–575 MB of disk; this overhead is benign on developer workstations but consequential in container-based scientific computing platforms (CI runners, JupyterHub spawners, edge devices) where image size and cold-start time are first-order constraints. SurtGIS can also be installed as a Python wheel (`pip install surtgis`, single command, ∼30 MB) without any `apt` dependencies, the recommended path for notebook users.

Position relative to service-publication frameworks. A complementary class of tools targets the *publication* of geospatial computations as networked services rather than the in-process execution model emphasised here. The OGC API—Processes specification (Open Geospatial Consortium, 2024) defines a REST contract for invoking server-side geoprocessing, and reference implementations include GeoServer WPS (GeoServer Project, 2025), pygeoapi (Kralidis et al., 2023), and PyWPS (PyWPS Project, 2025). Managed services such as the Sentinel Hub Processing API (Sinergise, 2025) and the Open Data Cube (Killough, 2018) expose curated raster archives with ana-

Table 17: Cold-start installation in fresh `ubuntu:24.04` Docker containers, from base image to working CLI invocation. "Wall" includes apt update, package downloads, and unpacking; "On-disk" is the size of the produced installation; "Apt deps" is the count of transitively installed Debian packages.

Tool	Method	Wall (s)	On-disk (MB)	Apt deps	System libraries
SurtGIS	release binary	15	22	0	none (static-linked)
SurtGIS	`pip install surtgis`	∼30	∼30	0	none
GDAL	`apt install gdal-bin`	29	315	197	libgdal, GEOS, PROJ, etc.
GRASS	`apt install grass-core`	36	575	242	GDAL, GEOS, PROJ, FFTW, NetCDF, BLAS, etc.
WBT[a]	`pip install whitebox`	20	26	112	python3-pip, libpython3

[a] WhiteboxTools downloads its ∼80 MB binary on first use; the on-disk figure here measures only the wrapper Python package. `cargo install surtgis` from source is also supported but requires installing a Rust toolchain (∼1 GB) and is therefore not directly comparable to the binary distributions.

lytic primitives over HTTP. SurtGIS occupies a different rung of the same stack: it is the *engine* of computation, not the *protocol* of publication. A SurtGIS algorithm can be wrapped behind an OGC API—Processes adapter and made remotely invocable, and conversely SurtGIS can be called from any of the above frameworks via the Python binding. The two models trade off latency against scalability: the in-process model eliminates the network round-trip and serialisation overhead that dominate short-running terrain operations on small DEMs, particularly under the WebAssembly target where the data never leaves the client (Section 5.3); the service-publication model amortises infrastructure across many tenants and avoids requiring users to install a runtime locally. We therefore view managed-service platforms as complementary, not competitive, with the SurtGIS computational core.

7. Discussion and conclusions

SurtGIS provides a high-performance, cross-platform geospatial analysis library that bridges the gap between computational efficiency and deployment flexibility. The systematic benchmarks presented in this paper demonstrate consistent performance advantages on fair end-to-end GeoTIFF pipelines at operationally relevant scales ($\geq$25 M cells): 1.7–1.8$\times$ over GDAL for slope and 1.5–2.0$\times$ for aspect, 3.5–4.9$\times$ over GRASS, and 3.2–7.9$\times$ over WhiteboxTools for aspect and hillshade (the WBT slope comparison is excluded because WBT uses a different derivative method; see Section 5.2). For hydrological algorithms, SurtGIS achieves 1.6–2.6$\times$ speedups for depression filling and 7.5–23.1$\times$ for D8 flow accumulation. GDAL retains an advantage for hillshade at $\geq$5 K^2, likely due to SIMD-optimized trigonometry. Numeri-

cal accuracy is within 5.7×10^{-5} degrees RMSE against analytical solutions, and the Python binding overhead of 2.7–3.9× relative to native Rust is competitive with single-threaded native execution.

Several design decisions and limitations merit discussion. First, SurtGIS operates by default on in-memory arrays rather than streaming from disk, which limits the in-memory model to available RAM (peak RSS 1.9–2.6 GB at $10K^2$; Supplementary Table S2); the v0.7 release ships out-of-core streaming variants for twelve terrain algorithms (Section 2.2), and broader streaming coverage is planned for future versions. Second, the TPI comparison highlights the importance of specifying algorithmic parameters when benchmarking: GDAL's 3×3 TPI and SurtGIS's radius-10 TPI are fundamentally different operations. Third, GDAL outperforms SurtGIS for hillshade at large DEM sizes, likely due to SIMD-optimized trigonometry; exploiting Rust's `std::arch` SIMD intrinsics is a targeted optimization opportunity.

The benchmarks have several limitations that should be considered when interpreting results. All measurements were collected on a single hardware platform (Intel i7-1270P with a heterogeneous architecture: 4 performance cores and 8 efficiency cores) under the default `powersave` CPU governor with Turbo Boost enabled. Speedup ratios may differ on other architectures (e.g., AMD Ryzen, ARM64 Apple Silicon) or systems with homogeneous core configurations, where Rayon's work-stealing scheduler may exhibit different scaling behavior. The scalability experiment uses synthetic fractal Brownian motion DEMs; real-world DEMs with irregular NoData patterns, extensive flat regions, or sensor artifacts may produce different performance characteristics, particularly for depression filling and flow routing algorithms. The use

of uncompressed Float32 GeoTIFFs eliminates codec-related variance but does not represent typical workflows where DEMs are LZW- or DEFLATE-compressed; SurtGIS's pure-Rust `tiff` crate decoder is significantly slower than GDAL's optimized C codecs for compressed data, which would reduce the observed speedup advantage in production settings.

The open-source implementation of Florinsky's complete 14-curvature morphometric system (Florinsky, 2025) removes a barrier to adoption of these theoretically superior curvature descriptors in environmental modeling. The integrated hydrological pipeline—from depression handling through flow direction to derived indices—provides a cohesive workflow without requiring users to chain multiple tools.

Validation coverage. The benchmarks and cross-tool comparisons presented here cover the most commonly used algorithms: slope, aspect, hillshade, depression filling (Priority-Flood), D8 and D-infinity flow direction, and flow accumulation. The 14 Florinsky curvatures are validated analytically on two surfaces (Gaussian hill and anisotropic sinusoidal ridge) and cross-validated against SAGA for the seven overlapping types. Several algorithms listed in Supplementary Table S1 have not been independently validated in this paper, including: MFD flow accumulation (only D8 was cross-compared), TFGA flow direction, Planchon–Darboux filling (only Priority-Flood was benchmarked), watershed delineation, and Strahler stream ordering. These algorithms pass internal unit tests (637 total across the test suite) but have not been compared against external reference implementations. Systematic cross-tool validation of these remaining algorithms is planned for future work.

Within the Python hydrological ecosystem, SurtGIS occupies a comple-

mentary niche. pysheds (Bartos, 2020) provides lightweight D8/D-infinity routing with efficient memory use, and pyflwdir (Eilander et al., 2021) offers fast pure-Python flow direction methods compatible with HydroSHEDS and MERIT Hydro datasets; both are well suited for interactive notebook workflows. SurtGIS offers a broader scope (terrain morphometry, depression handling, and derived indices alongside flow routing) and significantly higher throughput for large DEMs through compiled Rust code, but currently lacks native support for standard hydrological datasets such as HydroSHEDS or MERIT Hydro; users must load these as GeoTIFF via external tools. The `surtgis-cloud` crate provides experimental STAC catalog browsing and COG reading via HTTP range requests, but cloud-native hydrological workflows (e.g., streaming watershed analysis over COG tiles) are not yet implemented and remain a development target.

The cross-platform architecture enables new use cases for environmental modeling: browser-based teaching tools for terrain analysis, client-side processing of sensitive data that cannot leave the user's machine, and offline-capable field applications via Progressive Web Apps. However, the current WASM target is single-threaded, which limits practical browser-based processing to DEMs up to approximately $1K^2$–$2K^2$ cells for interactive use; a complete hydrological workflow (fill + flow direction + accumulation + TWI) on a $5K^2$ DEM would take >15 s in the browser (extrapolated from Table 10). The Python bindings make SurtGIS accessible to the large scientific Python ecosystem while preserving the performance benefits of the native Rust implementation.

Software sustainability. SurtGIS is currently developed and maintained by a single author (latest release v0.7.0 at the time of writing), which raises legitimate concerns about long-term sustainability compared to established projects such as GDAL (>100 contributors, 20+ years of development) or GRASS GIS (active since 1982). To mitigate the single-developer risk: (a) the project uses permissive dual licensing (MIT/Apache-2.0) to lower the barrier for external contributions; (b) continuous integration enforces code quality across 9 automated jobs including cross-platform builds; (c) the modular crate architecture allows contributors to work on individual components without understanding the full codebase; and (d) the Rust API follows semantic versioning, with the 0.x series indicating that breaking changes may occur before a 1.0 stability commitment. Community adoption is in its early stages; building a contributor base and establishing governance structures are priorities for the project's next phase.

Future development will focus on: (1) expanding WebAssembly support to include multi-threaded execution via SharedArrayBuffer; (2) extending streaming I/O coverage to the hydrology and imagery modules (the v0.7 release ships streaming variants for twelve terrain algorithms; broader coverage will lift the in-memory size limit for the remaining algorithm classes); (3) parallel tile-and-reconcile variants of Priority-Flood and D8 flow accumulation following Barnes (2017) to break the Amdahl ceiling identified in Experiment 4; (4) GPU acceleration for computationally intensive operations via WebGPU; and (5) maturing the STAC + COG cloud workflow stack toward streaming watershed analysis directly over COG tiles.

Software availability

Name of software: SurtGIS

Developers: Francisco Parra

Contact: francisco.parra.o@usach.cl

Year first available: 2026

Hardware required: Any x86_64 or ARM64 system; WebAssembly target requires a modern browser

Software required: Rust 1.75+ (compilation); Python 3.8+ (Python bindings); Node.js 16+ (npm package)

Program language: Rust

Program size: ∼48 000 lines of Rust source code (33 K algorithms, 6 K GUI, 3.8 K cloud, 1.7 K core, 1.5 K CLI, 0.9 K Python bindings, 0.4 K WASM bindings, 0.4 K parallel; plus ∼1 K tests and ∼0.8 K examples)

Test suite: 637 unit and integration tests; CI via GitHub Actions (9 jobs: check, clippy, test with/without parallel, WASM build, web demo build, Python build, format check, integration tests on real DEM, benchmark compilation)

Availability: Open source under MIT/Apache-2.0 dual license

Source code: `https://github.com/franciscoparrao/surtgis`

Rust crates: `https://crates.io/crates/surtgis-core`

Python package: `https://pypi.org/project/surtgis/`

npm package: `https://www.npmjs.com/package/surtgis`

CRediT authorship contribution statement

Francisco Parra: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

All source code, benchmark scripts, raw benchmark results (CSV), and synthetic DEM generation code are available in the project repository (`https://github.com/franciscoparrao/surtgis`). The scripts and raw results reproducing the comparative experiments added in this revision (Sections 5.5, 5.4.1, and Table 5) are additionally archived at Zenodo, `https://doi.org/10.5281/zenodo.20836786`. The Copernicus 30 m DEM used in the case study is freely available from the Copernicus Climate Data Store.

Funding

This research was supported by the DICYT Postdoctoral Programme of the Universidad de Santiago de Chile (Project 062619MC_POSTDOC), through the Vicerrectoría de Investigación, Innovación y Creación (VRIIC).

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author used Claude (Anthropic) for code scaffolding, debugging assistance, benchmark automation scripting, and manuscript drafting. All algorithm designs (Florinsky curvatures, flow direction methods, depression handling) and architectural decisions (workspace layout, `maybe_rayon` pattern, cross-platform compilation strategy) were conceived by the author. All figures were generated programmatically using matplotlib or TikZ; no generative AI was used in the production of artwork. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

Acknowledgements

The author thanks the Rust geospatial community for foundational crates including `ndarray`, `geo`, and `proj`, and the Vicerrectoría de Investigación, Innovación y Creación of Universidad de Santiago de Chile for support through the DICYT Postdoctoral Programme.

References

Amatulli, G., Domisch, S., Tuanmu, M.N., Parmentier, B., Ranipeta, A., Malczyk, J., Jetz, W., 2018. A suite of global, cross-scale topographic variables for environmental and biodiversity modeling. Scientific Data 5, 180040. doi:10.1038/sdata.2018.40.

Barnes, R., 2016. RichDEM: Terrain analysis software. URL: https://github.com/r-barnes/richdem. accessed: 2026-02-12.

Barnes, R., 2017. Parallel non-divergent flow accumulation for trillion cell digital elevation models on desktops or clusters. Environmental Modelling & Software 92, 202–212. doi:10.1016/j.envsoft.2017.02.022.

Barnes, R., Lehman, C., Mulla, D., 2014. Priority-flood: An optimal depression-filling and watershed-labeling algorithm for digital elevation models. Computers & Geosciences 62, 117–127. doi:10.1016/j.cageo.2013.04.024.

Bartos, M., 2020. pysheds: Simple and fast watershed delineation in python. URL: https://github.com/mdbartos/pysheds. accessed: 2026-02-16.

Blumofe, R.D., Leiserson, C.E., 1999. Scheduling multithreaded computations by work stealing. Journal of the ACM 46, 720–748. doi:10.1145/324133.324234.

Conrad, O., Bechtel, B., Bock, M., Dietrich, H., Fischer, E., Gerlitz, L., Wehberg, J., Wichmann, V., Böhner, J., 2015. System for automated geoscientific analyses (SAGA) v. 2.1.4. Geoscientific Model Development 8, 1991–2007. doi:10.5194/gmd-8-1991-2015.

De Reu, J., Bourgeois, J., Bats, M., Zwertvaegher, A., Gelorini, V., De Smedt, P., Chu, W., Antrop, M., De Maeyer, P., Finke, P., Van Meirvenne, M., Verniers, J., Crombé, P., 2013. Application of the topographic position index to heterogeneous landscapes. Geomorphology 186, 39–49. doi:10.1016/j.geomorph.2012.12.015.

Eilander, D., van Verseveld, W., Winsemius, H., Ward, P., 2021. pyflwdir: Fast methods to work with hydro- and topography data in pure Python. URL: https://github.com/Deltares/pyflwdir, doi:10.5281/zenodo.6108034.

Evans, I.S., 1972. General geomorphometry, derivatives of altitude, and descriptive statistics, in: Chorley, R.J. (Ed.), Spatial Analysis in Geomorphology. Methuen, London, pp. 17–90. doi:10.4324/9780429273346-2.

Farr, T.G., Rosen, P.A., Caro, E., Crippen, R., Duren, R., Hensley, S., Kobrick, M., Paller, M., Rodriguez, E., Roth, L., Seal, D., Shaffer, S., Shimada, J., Umland, J., Werner, M., Oskin, M., Burbank, D., Alsdorf, D., 2007. The Shuttle Radar Topography Mission. Reviews of Geophysics 45, RG2004. doi:10.1029/2005RG000183.

Florinsky, I.V., 2016. Digital Terrain Analysis in Soil Science and Geology. 2nd ed., Academic Press. doi:10.1016/C2015-0-01431-6.

Florinsky, I.V., 2025. Digital Terrain Analysis. 3rd ed., Elsevier. doi:10.1016/C2022-0-02025-4.

GDAL/OGR contributors, 2024. GDAL/OGR geospatial data abstraction software library. URL: https://gdal.org, doi:10.5281/zenodo.5884351.

GeoServer Project, 2025. GeoServer Web Processing Service (WPS). URL: https://docs.geoserver.org/latest/en/user/services/wps/.

Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., Moore, R., 2017. Google Earth Engine: Planetary-scale geospatial analysis for everyone. Remote Sensing of Environment 202, 18–27. doi:10.1016/j.rse.2017.06.031.

Guan, Q., Clarke, K.C., 2010. A general-purpose parallel raster processing programming library test application using a geographic cellular automata model. International Journal of Geographical Information Science 24, 695–722. doi:10.1080/13658810902984228.

Haas, A., Rossberg, A., Schuff, D.L., Titzer, B.L., Holman, M., Gohman, D., Wagner, L., Zakai, A., Bastien, J., 2017. Bringing the web up to speed with WebAssembly, in: Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, ACM. pp. 185–200. doi:10.1145/3062341.3062363.

Horn, B.K.P., 1981. Hill shading and the reflectance map. Proceedings of the IEEE 69, 14–47. doi:10.1109/PROC.1981.11918.

Jasiewicz, J., Stepinski, T.F., 2013. Geomorphons — a pattern recognition approach to classification and mapping of landforms. Geomorphology 182, 147–156. doi:10.1016/j.geomorph.2012.11.005.

Killough, B., 2018. Overview of the Open Data Cube initiative. IGARSS 2018 – IEEE International Geoscience and Remote Sensing Symposium , 8629–8632doi:10.1109/IGARSS.2018.8517694.

Kopecký, M., Macek, M., Wild, J., 2021. Topographic wetness index calculation guidelines based on measured soil moisture and plant species composition. Science of The Total Environment 757, 143785. doi:10.1016/j.scitotenv.2020.143785.

Kralidis, T., Tzotsos, A., pygeoapi contributors, 2023. pygeoapi: an OGC API reference implementation. OSGeo Journal URL: https://pygeoapi.io.

Li, Z., Lai, X., Shi, P., Yang, T., 2024. Proposal of a facet-to-facet multiple flow direction algorithm based on geometrical and mathematical analysis of physical dispersion over triangle facet. Geomorphology 466, 109440. doi:10.1016/j.geomorph.2024.109440.

Lindsay, J.B., 2016a. Efficient hybrid breaching-filling sink removal methods for flow path enforcement in digital elevation models. Hydrological Processes 30, 846–857. doi:10.1002/hyp.10648.

Lindsay, J.B., 2016b. Whitebox GAT: A case study in geomorphometric analysis. Computers & Geosciences 95, 75–84. doi:10.1016/j.cageo.2016.07.003.

Miao, J., Guan, Q., Hu, S., 2017. pRPL+GTIOL: the marriage of a parallel processing library and a parallel i/o library for big raster data. Environmental Modelling & Software 96, 347–360. doi:10.1016/j.envsoft.2017.07.001.

Minár, J., Evans, I.S., Jenčo, M., 2020. A comprehensive system of definitions of land surface (topographic) curvatures, with implications for their

application in geoscience modelling and prediction. Earth-Science Reviews 211, 103414. doi:10.1016/j.earscirev.2020.103414.

Neteler, M., Bowman, M.H., Landa, M., Metz, M., 2012. GRASS GIS: A multi-purpose open source GIS. Environmental Modelling & Software 31, 124–130. doi:10.1016/j.envsoft.2011.11.014.

Newman, D.R., Lindsay, J.B., Cockburn, J.M.H., 2018. Evaluating metrics of local topographic position for multiscale geomorphometric analysis. Geomorphology 312, 40–50. doi:10.1016/j.geomorph.2018.04.003.

Nobre, A.D., Cuartas, L.A., Hodnett, M., Rennó, C.D., Rodrigues, G., Silveira, A., Waterloo, M., Saleska, S., 2011. Height above the nearest drainage – a hydrologically relevant new terrain model. Journal of Hydrology 404, 13–29. doi:10.1016/j.jhydrol.2011.03.051.

O'Callaghan, J.F., Mark, D.M., 1984. The extraction of drainage networks from digital elevation data. Computer Vision, Graphics, and Image Processing 28, 323–344. doi:10.1016/S0734-189X(84)80011-0.

Open Geospatial Consortium, 2024. OGC API — Processes — Part 1: Core. URL: https://docs.ogc.org/is/18-062r2/18-062r2.html. standard 18-062r2.

Perkel, J.M., 2020. Why scientists are turning to Rust. Nature 588, 185–186. doi:10.1038/d41586-020-03382-2.

Planchon, O., Darboux, F., 2001. A fast, simple and versatile algorithm to fill the depressions of digital elevation models. Catena 46, 159–176. doi:10.1016/S0341-8162(01)00164-3.

PyO3 Project Contributors, 2024. PyO3: Rust bindings for Python. URL: https://github.com/PyO3/pyo3. version 0.22. Accessed: 2026-02-16.

PyWPS Project, 2025. PyWPS: Python implementation of the OGC web processing service standard. URL: https://pywps.org.

Qin, C.Z., Zhan, L.J., Zhu, A.X., 2014. A strategy for raster-based geocomputation under different parallel computing platforms. International Journal of Geographical Information Science 28, 2127–2144. doi:10.1080/13658816.2014.911300.

Quinn, P., Beven, K., Chevallier, P., Planchon, O., 1991. The prediction of hillslope flow paths for distributed hydrological modelling using digital terrain models. Hydrological Processes 5, 59–79. doi:10.1002/hyp.3360050106.

Radiant Earth Foundation, 2021. SpatioTemporal Asset Catalog (STAC) specification. URL: https://stacspec.org. version 1.0.0.

Sinergise, 2025. Sentinel Hub Processing API documentation. URL: https://docs.sentinel-hub.com/api/latest/api/process/.

Tarboton, D.G., 1997. A new method for the determination of flow directions and upslope areas in grid digital elevation models. Water Resources Research 33, 309–319. doi:10.1029/96WR03137.

Tarboton, D.G., 2015. TauDEM – Terrain Analysis Using Digital Elevation Models. URL: https://hydrology.usu.edu/taudem/. utah State University. Accessed: 2026-02-16.

1364 Wang, Y., Ai, B., Qin, C.Z., 2021. A load-balancing strategy for data domain
1365 decomposition in parallel programming libraries of raster-based geocom-
1366 putation. International Journal of Geographical Information Science 36,
1367 968–991. doi:10.1080/13658816.2021.2004603.

1368 Wilson, J.P., Gallant, J.C. (Eds.), 2000. Terrain Analysis: Principles and
1369 Applications. John Wiley & Sons, New York.